VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

Think Like a Senior Production Engineer

200 Linux Production Interview Questions & Answers

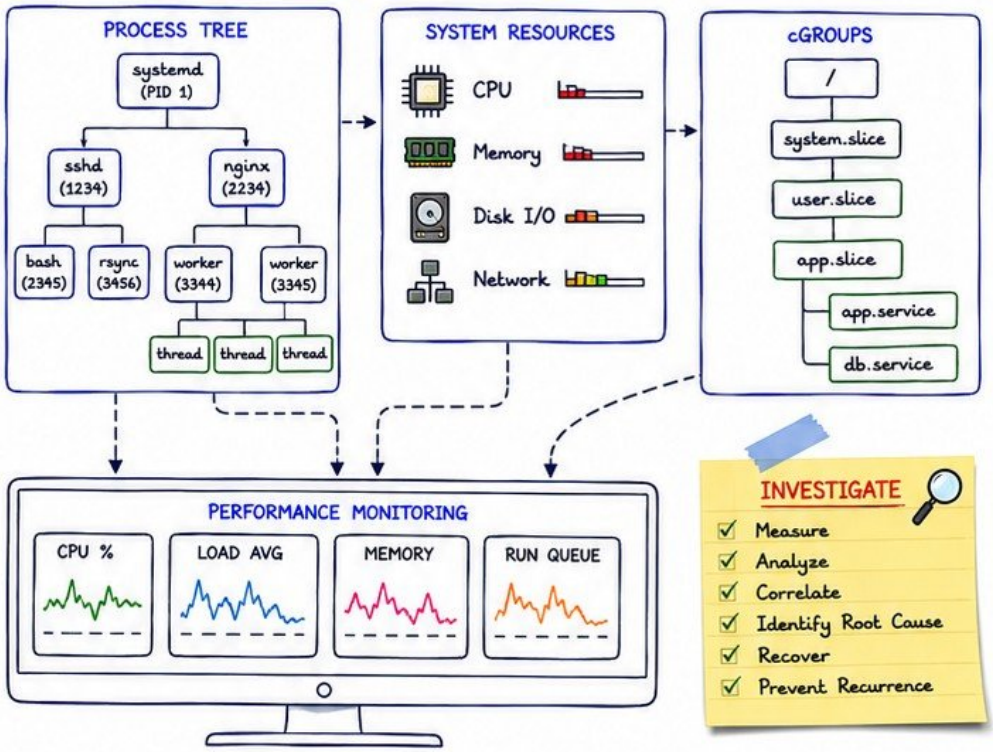


SECTION 2 PROCESSES, SERVICES AND PERFORMANCE

Pages 41 to 60 | Questions 201 to 300

COVERS

- ✓ Advanced Process Investigation
- ✓ Process Trees & Namespaces
- ✓ Zombies & Orphan Processes
- ✓ systemd Unit Files
- ✓ Service Dependencies
- ✓ Resource Limits & ulimit
- ✓ cgroups & Resource Governance
- ✓ CPU Scheduling & Priorities
- ✓ Load Average & Run Queue
- ✓ Memory Pressure & OOM Killer
- ✓ CPU Bottlenecks
- ✓ Context Switching
- ✓ NUMA & Multi-Socket Systems
- ✓ Performance Baselines
- ✓ top, htop, ps, pidstat
- ✓ vmstat, mpstat, sar
- ✓ Service Failure Scenarios
- ✓ Production Performance Incidents
- ✓ Senior Troubleshooting Challenges



INVESTIGATE

- ☑ Measure
- ☑ Analyze
- ☑ Correlate
- ☑ Identify Root Cause
- ☑ Recover
- ☑ Prevent Recurrence

Real-World Production Scenarios

Advanced Troubleshooting Techniques

Performance Analysis Mastery

Senior Engineer Level Content

Command Reference Included

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

ADVANCED PROCESS INVESTIGATION

QUESTIONS 201-205

201 How do you identify the process responsible for a production issue ?

202 How do you inspect a process beyond its name and PID ?

203 How do you determine which files and sockets a process has open ?

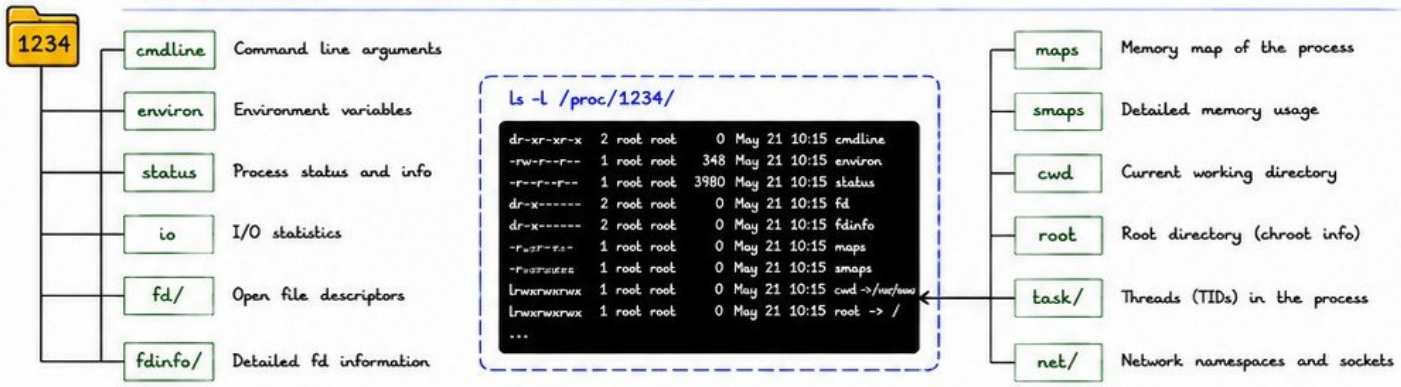
204 How do you investigate a process that starts and disappears immediately ?

205 How do you trace the activity of a running process safely ?

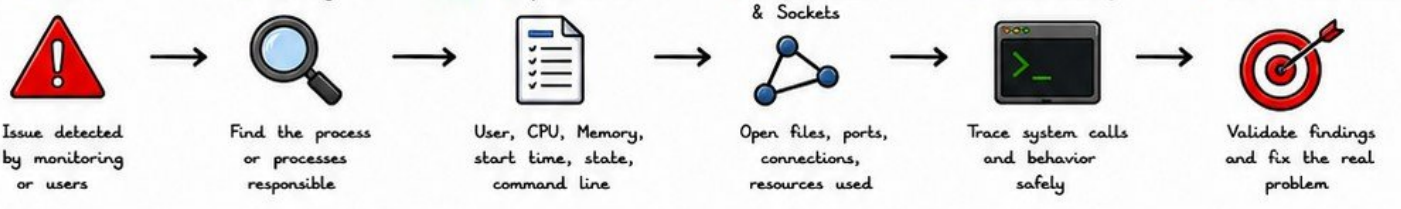
COMMANDS AND TOOLS

ps aux	Snapshot of all processes (BSD format)
ps -ef	Full format listing of all processes
ps -eo pid,ppid,user,stat,,%cpu,%mem,etime,cmd	Custom columns for targeted investigation
pgrep <pattern>	Find PID(s) by name pattern
pidof <process>	Return PID(s) of a running process
lsof -p <PID>	List open files and sockets for PID
strace -p <PID>	Trace system calls of a running process
/proc/<PID>/	Live process information in the kernel

/PROC/<PID>/ IMPORTANT DIRECTORIES AND FILES



PRODUCTION INVESTIGATION WORKFLOW



KEY INVESTIGATION TIPS

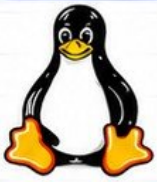
- ☑ Always identify the impact before digging in.
- ☑ Start with read-only commands.
- ☑ Collect facts before making changes.
- ☑ Correlate CPU, memory, I/O, and logs.
- ☑ Restart or kill only after you understand WHY.
- ☑ Document everything for post-incident review.

COMMON INVESTIGATION PATTERNS

- High CPU → top/htop → ps → perf / strace
- High Memory → free → smem → lsof → /proc
- Too many open files → lsof → limits → fix app
- Service crash loop → journalctl → strace → fix
- Port in use → lsof -i → identify process → fix

CAUTION

- ✓ strace can generate high overhead.
- ✓ lsof on busy systems can be heavy.
- ✓ Do not kill critical processes blindly.
- ✓ Understand dependencies before action.
- ✓ Use maintenance windows when possible.



VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

PROCESS TREES AND PARENT RELATIONSHIPS

QUESTIONS 206-210

206 What does a Linux process tree reveal during troubleshooting?

207 How do parent and child processes affect application recovery?

208 How do you identify the parent responsible for repeatedly restarting a failed process?

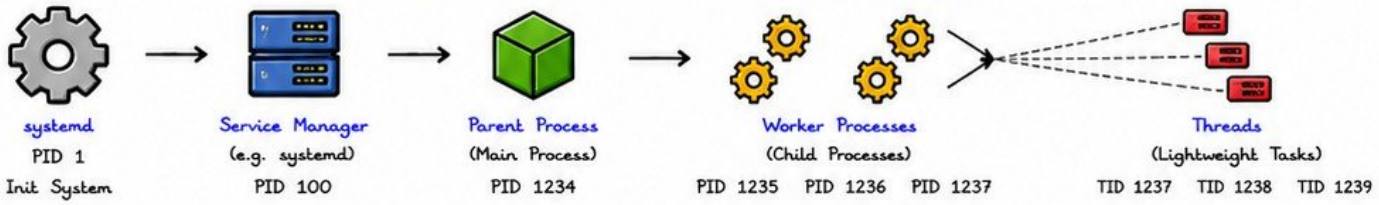
209 What happens when a parent process terminates before its children?

210 How do you investigate unexpectedly duplicated process trees?

COMMANDS AND TOOLS

pstree -p	Display process tree with PIDs
ps --forest	Show process hierarchy in forest format
ps -o pid,ppid,stat,cmd	Show PID, PPID, state and command
pgrep -P <PPID>	Find child processes of a parent
/proc/<PID>/status	Detailed status and metadata of a process
/proc/<PID>/task	List all threads (TIDs) of a process

PROCESS TREE DIAGRAM (TYPICAL LINUX SERVICE)



EXAMPLE: pstree -p OUTPUT

```
systemd(1)
├─nginx(1234)
│   ├─nginx(1235)
│   ├─nginx(1236)
│   └─nginx(1237)
└─mysqld(2345)
    ├─mysqld(2346)
    └─mysqld(2347)
```

systemd(1) ⟷ Init system (PID 1)

nginx(1234) ⟷ Main Nginx process (parent)

nginx(1235), nginx(1236), nginx(1237) → Worker processes (children)

mysqld(2345) ⟷ Main MySQL process (parent)

mysqld(2346), mysqld(2347) → Worker processes (children)

EXAMPLE: ps --forest OUTPUT (TRUNCATED)

UID	PID	PPID	STAT	CMD
root	1	0	Ss	/sbin/init
root	100	1	S	/lib/systemd/systemd
root	1234	100	S	nginx: master process
_ www-data	1235	1234	S	nginx: worker process
_ www-data	1236	1234	S	nginx: worker process
_ www-data	1237	1234	S	nginx: worker process
mysql	2345	100	S	/usr/sbin/mysqld
_ mysql	2346	2345	S	/usr/sbin/mysqld
_ mysql	2347	2345	S	/usr/sbin/mysqld

KEY CONCEPTS

- ☑ Every process (except PID 1) has a parent (PPID).
- ☑ Children inherit the environment of their parent.
- ☑ Parent process controls the lifecycle of its children.
- ☑ If parent dies, children are adopted by PID 1 (systemd).
- ☑ Thread IDs (TIDs) are found under /proc/<PID>/task.
- ☑ Understanding the tree helps find the real root cause.

INVESTIGATION WORKFLOW

1. Identify the problematic process (PID).
2. View the process tree (pstree -p or ps --forest).
3. Find the parent (PPID).
4. Inspect parent behavior and restart patterns.
5. Check children and threads for impact.
6. Gather details using /proc/<PID>/status.
7. Correlate logs and confirm root cause.

COMMON PATTERNS TO LOOK FOR

- Parent repeatedly spawning a failing child.
- Orphaned children adopted by systemd (PPID 1).
- Many duplicate processes from same parent.
- Worker processes growing abnormally.
- Threads (TIDs) consuming high resources.

PRO TIP

Always follow the tree UPWARDS to find the controller, and DOWNWARDS to understand the impact. The parent is often the key to the root cause.

ONE-LINERS

- Show tree for a PID: `pstree -p <PID>`
- Find children of PID: `pgrep -P <PID>`
- Show PPID and command: `ps -o pid,ppid,stat,cmd -p <PID>`
- Show threads of PID: `ls /proc/<PID>/task`

CAUTION

- ✓ Killing the wrong parent can stop healthy services.
- ✓ Unexpected duplicates may cause port conflicts.
- ✓ Threads can be the real CPU or memory hogs.
- ✓ Always capture evidence before taking action.

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

LINUX NAMESPACES AND PROCESS ISOLATION

QUESTIONS 211-215

211 What are Linux namespaces, and why do production systems use them ?

212 Which namespaces isolate processes, networks, mounts, users, and hostnames ?

213 How do you identify the namespaces used by a running process ?

214 How can two processes see different filesystems or network interfaces on the same host ?

215 How do namespaces support container isolation ?

NAMESPACES OVERVIEW

NAMESPACE	ISOLATES	WHAT ISOLATED?	TYPICAL USE
PID	Process IDs	Process ID space	Containers, sandboxes
Mount	Mount points	Filesystem mount view	Containers, chroot jails
Network	Network stack	Interfaces, IPs, routes, ports, firewall	Containers, network virtualization
User	User and group IDs	UIDs, GIDs, capabilities	Rootless containers, security isolation
UTS	Hostname and domain	Hostname, NIS domain	Containers, multi-tenant applications
IPC	IPC resources	System V IPC, POSIX message queues, semaphores	Containers, app isolation
Cgroup	Resource control	cgroups hierarchy and controllers	Containers, resource isolation
Time	System time	Boot time, monotonic clock, time offsets	Containers, testing, time simulation

COMMANDS AND TOOLS

lsns — List all namespaces on the system

```
$ lsns
        TYPE NPROCS   PID USER COMMAND
4026531837 mnt     214     1 root /sbin/init
4026531838 uts     214     1 root /sbin/init
4026531839 ipc     214     1 root /sbin/init
4026531840 pid     214     1 root /sbin/init
4026531956 net     214     1 root /sbin/init
4026532810 user    214     1 root /sbin/init
4026532811 cgroup  214     1 root /sbin/init
4026532812 time      1 32768 sleep 1000
4026532813 time    214     1 root /sbin/init
```

nsenter — Enter a namespace of a running process

```
# Enter the mount namespace of PID 1234
$ sudo nsenter -t 1234 -m /bin/bash

# Enter network namespace of PID 1234
$ sudo nsenter -t 1234 -n /bin/bash
```

Common options:

-m	Mount namespace	-p	PID namespace
-n	Network namespace	-U	User namespace
-u	UTS namespace	-T	Time namespace
-i	IPC namespace	-a	Enter all namespaces

unshare — Run a process in new (unshared) namespaces

```
# Run a shell in new PID and mount namespaces
$ unshare -pfm /bin/bash

# Run a shell in a new network namespace
$ unshare -n /bin/bash
```

Common options:

-m	Mount	-i	IPC
-n	Network	-U	User
-u	UTS	-T	Time
-p	PID	-a	All namespaces

readlink /proc/<PID>/ns/* — Show namespaces used by a specific process

```
$ readlink /proc/1234/ns/*
ipc -> ipc:[4026531839]
mnt -> mnt:[4026531837]
net -> net:[4026531956]
pid -> pid:[4026531840]
user -> user:[4026532810]
uts -> uts:[4026531838]
cgroup -> cgroup:[4026532811]
time -> time:[4026532812]
```

TIP

Same inode number = same namespace

- ✓ Different inode number = different namespace
- ✓ Namespace inode numbers are unique identifiers
- ✓ Containers have their own namespace inodes

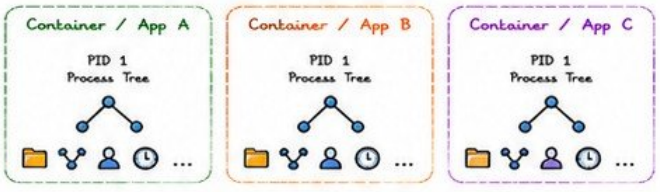
NOTE

Namespaces don't virtualize the kernel. They provide isolated *views* of the kernel resources for each process.

NAMESPACE ISOLATION DIAGRAM



ISOLATED PROCESS ENVIRONMENTS



Each environment has its own isolated view of processes, filesystem, network, users, time, etc.

WHY NAMESPACES MATTER IN PRODUCTION

- ✓ Run multiple isolated applications on the same host.
- ✓ Prevent one application from seeing another's processes.
- ✓ Provide secure boundaries for containers and microservices.
- ✓ Support multi-tenancy and resource isolation.
- ✓ Enable advanced troubleshooting without system impact.

COMMON USE CASES

- Containers (Docker, Podman, Kubernetes)
- Sandboxing untrusted applications
- Rootless containers (user namespace)
- Network isolation for testing
- Application isolation on shared servers

BEST PRACTICES

- ✓ Understand which namespaces your containers are using.
- ✓ Use rootless containers when possible.
- ✓ Verify isolation with lsns and readlink.
- ✓ Avoid sharing host resources unnecessarily.
- ✓ Monitor resource usage with cgroups.



VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

ZOMBIE, ORPHAN AND STUCK PROCESSES

Think Like a Senior Production Engineer

QUESTIONS 216-220

216 How is a zombie process created ?

217 Why can a zombie not be removed with kill -9 ?

218 When do zombie processes become a production risk ?

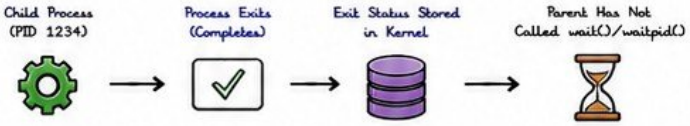
219 How does Linux adopt an orphan process ?

220 How do you investigate a process stuck in uninterruptible sleep ?

PROCESS STATES

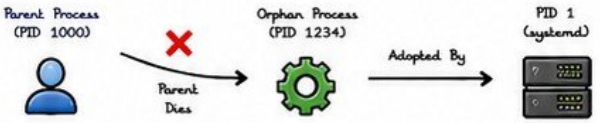
STATE	CODE	DESCRIPTION	HOW TO IDENTIFY
Running	R	Actively using CPU	ps/top (STAT = R)
Interruptible Sleep	S	Waiting for an event (can be interrupted)	ps/top (STAT = S)
Uninterruptible Sleep	D	Waiting for I/O or kernel resource (cannot be killed)	ps/top (STAT = D)
Stopped	T	Stopped by signal or job control	ps/top (STAT = T)
Zombie	Z	Process has finished but parent didn't collect exit status	ps/top (STAT = Z)
Idle Kernel Thread	I	Kernel thread, no user process	ps/top (STAT = I)

WHAT IS A ZOMBIE PROCESS?



The process is dead, but its entry remains in the process table until the parent collects the exit status.

HOW ORPHAN PROCESSES ARE ADOPTED



When the parent dies, PID 1 (systemd) becomes the new parent and is responsible for reaping it.

COMMANDS AND TOOLS

COMMAND	DESCRIPTION	EXAMPLE	WHAT TO LOOK FOR
`ps -eo pid,ppid,stat,wchan,cmd`	Detailed process list with state and wait channel	`ps -eo pid,ppid,stat,wchan,cmd | grep -E "Z|D"`	STAT = Z (zombie) or D (uninterruptible) PPID shows the parent PID
`pstree -p`	Display process tree with PIDs	`pstree -p | less`	Look for <defunct> (zombie) or large trees
`cat /proc/<PID>/status`	Detailed info about a process	`cat /proc/1234/status`	State:, PPid:, NStgid:, Threads:
`cat /proc/<PID>/stack`	Kernel stack trace (for D state)	`cat /proc/1234/stack`	Shows where the process is stuck in kernel
`dmesg`	Kernel ring buffer logs	`dmesg | grep -i -E "oom|blocked|error"`	Look for I/O errros, blocked devices, oom
`strace -p <PID>`	Trace system calls of a running process	`strace -p 1234`	Useful if process is in S (sleep) state

TIP: Zombie processes show STAT=Z and <defunct> in the CMD column. Processes in D state will not respond to kill -9 because they are stuck in the kernel.

PRODUCTION SCENARIO

A server has hundreds of zombie processes, but CPU and memory usage remain normal.

IMPACT:

- ☑ Process table fills up over time
- ☑ New processes may fail to start (error: process table full)
- ☑ Monitoring becomes noisy
- ☑ Indicates a bug in parent process or improper wait handling

INVESTIGATION WORKFLOW

1. Check for zombies
   ```
   ps -eo pid,ppid,stat,cmd | awk '$3 ~ /^Z/'
   ```
2. Find the parent process
   ```
   ps -o pid,ppid,cmd -p <ZOMBIE_PID>
   ```
3. Inspect parent process
   ```
   cat /proc/<PPID>/status
   ```
4. Check parent children
   ```
   pstree -p <PPID>
   ```
5. Fix the root cause
   - Ensure parent processes call wait()/waitpid()
   - Restart misbehaving parent if required
   - Monitor to prevent recurrence

INVESTIGATING D STATE (UNINTERRUPTIBLE SLEEP)

1. Identify D state processes
   ```
   ps -eo pid,ppid,stat,wchan,cmd | grep ^D
   ```
2. Check what they are waiting for
   ```
   cat /proc/<PID>/stack
   ```
3. Check kernel logs
   ```
   dmesg | tail -n 50
   ```
4. Common causes
   - Blocked disk I/O
   - NFS/remote filesystem issue
   - Database or storage lock
   - Hardware or driver problems

WHY KILL -9 DOES NOT REMOVE ZOMBIE

A zombie is already dead. Only its parent can remove it by calling wait() or waitpid(). Since zombies are not running, signals like kill -9 have no effect.

ORPHAN VS ZOMBIE

ORPHAN	ZOMBIE
• Parent died	• Process finished
• Adopted by PID 1	• Parent still alive
• Still running	• Exit status not collected
• Will be reaped by PID 1	• Stays in process table

BEST PRACTICES

- ✓ Always handle child processes properly
- ✓ Use wait()/waitpid() in scripts and apps
- ✓ Monitor for zombie and D state processes
- ✓ Investigate and fix root cause early
- ✓ Keep kernel and drivers up to date

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

SYSTEMD UNIT FILE ARCHITECTURE

QUESTIONS 221-225

221 What sections make up a systemd service unit file ?

222 What is the purpose of the [Unit], [Service], and [Install] sections ?

223 How do Type=simple, forking, oneshot, and notify differ ?

224 How do you create and deploy a custom systemd service ?

225 Why must systemctl daemon-reload be run after changing a unit file ?

UNIT FILE SECTIONS EXPLAINED

SECTION	PURPOSE	EXAMPLE KEYS
[Unit]	Defines metadata, dependencies, and ordering.	Description, After, Requires, Wants, Conflicts, ConditionPathExists
[Service]	Defines how the service starts, stops, runs and is managed.	Type, ExecStart, ExecReload, ExecStop, User, Group, Restart, WorkingDirectory, EnvironmentFile
[Install]	Defines how the service is enabled and its target.	WantedBy, RequiredBy, Alias

COMMON SERVICE TYPES

TYPE	DESCRIPTION	HOW IT WORKS
simple	Default. Service runs in foreground.	ExecStart is the main process. systemd considers it started immediately.
forking	Service forks to background (creates child).	Parent exits, child continues. PIDFile is usually used.
oneshot	Runs a short task and exits.	ExecStart runs and exits. Service considered active after completion.
notify	Service notifies systemd when ready.	Service must call sd-notify() to signal readiness.

EXAMPLE: /etc/systemd/system/myapp.service

```
[Unit]
Description=MyApp Service
After=network.target                          # Matadata and dependencies
Wants=network-online.target

[Service]
Type=simple
User=myapp
Group=myapp                                   # How the service runs
WorkingDirectory=/opt/myapp                     and is managed
EnvironmentFile=/etc/myapp/myapp.env
ExecStart=/opt/myapp/bin/myapp --config /etc/myapp/config.yml
ExecReload=/bin/kill -HUP $MAINPID
ExecStop=/bin/kill -TERM $MAINPID
Restart=on-failure
RestartSec=5s

[Install]                                     # Where the service
WantedBy=multi-user.target                      should be enabled
```

KEY UNIT FILE ELEMENTS

- Description – Human readable description.
- After – Start this unit after listed units.
- Requires – Fail if the required unit fails.
- Wants – Start this unit if available (non-fatal).
- ExecStart – Command to start the service.
- ExecReload – Command to reload the service.
- ExecStop – Command to stop the service.
- User – Run the service as this user.
- Group – Run the service with this group.
- WorkingDirectory – Set working directory before start.
- EnvironmentFile – Load environment variables from file.
- Restart – Restart policy (on-failure, always, etc.).
- WantedBy – Target that pulls in this service when enabled.

SYSTEMD SERVICE WORKFLOW

1. Create Unit File — Write the unit file in /etc/systemd/system/
2. Validate Syntax — systemd-analyze verify myapp.service or systemctl cat myapp.service
3. Reload systemd — systemctl daemon-reload (IMPORTANT)
4. Enable Service — systemctl enable myapp.service Creates symlink in /etc/systemd/system/
5. Start Service — systemctl start myapp.service
6. Check Status and Logs — systemctl status myapp.service journalctl -u myapp.service -f

WHY SYSTEMCTL DAEMON-RELOAD IS REQUIRED

systemd keeps a cached copy of all unit files in memory. When you create, edit or delete a unit file, systemd does not automatically re-read the changes.

daemon-reload tells systemd to:

- ✓ Re-scan unit files on disk
- ✓ Update the in-memory unit cache
- ✓ Apply the latest configuration

COMMON UNIT FILE LOCATIONS

- /etc/systemd/system/ — Custom admin created units
- /usr/lib/systemd/system/ — Vendor packages (do not edit)
- /run/systemd/system/ — Runtime generated units
- /etc/systemd/system/multi-user.target.wants/ — Symlinks for enabled services

BEST PRACTICES

- ✓ Always set User and Group for security.
- ✓ Use Restart=on-failure for most services.
- ✓ Set RestartSec to avoid restart storms.
- ✓ Use Environment=File for configuration.
- ✓ Validate unit files before deploying.
- ✓ Keep unit files simple and readable.
- ✓ Use meaningful Description.
- ✓ Monitor logs after deployment.

PRODUCTION TIP

A well-designed unit file improves reliability, observability and maintainability of production services.

Quick Check

- ☑ Unit file syntax valid ?
- ☑ Dependencies correct ?
- ☑ Permissions correct ?
- ☑ Service starts cleanly ?

COMMON MISTAKES

- ✗ Forgetting daemon-reload after changes.
- ✗ Wrong Type= causing service not to start.
- ✗ Missing After= leading to race conditions.
- ✗ Using Restart=always without limits.

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

SERVICE DEPENDENCIES AND STARTUP ORDERING

QUESTIONS 226–230

226 What is the difference between Requires, Wants, Before, and After ?

227 Why does startup ordering not always create a dependency ?

228 How do you inspect the complete dependency tree of a service ?

229 How do dependency cycles affect system startup ?

230 How do you design a service that waits for network or storage readiness ?

REQUIRES, WANTS, BEFORE, AFTER — WHAT THEY MEAN

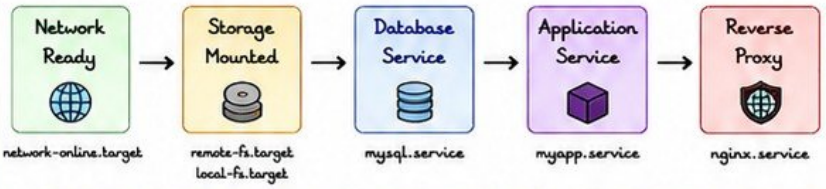
DIRECTIVE	TYPE	PURPOSE	EXAMPLE
Requires=	Dependency	If the required unit fails or is not started, this unit will fail.	Requires=network-online.target
Wants=	Weak Dependency	Requests the unit to start, but this unit can still start if it is not available.	Wants=sshd.service
Before=	Ordering	Ensures this unit starts before the specified unit.	Before=nginx.service
After=	Ordering	Ensures this unit starts after the specified unit.	After=network-online.target

KEY POINT: Ordering (Before/After) controls sequence. Dependencies (Requires/Wants) control necessity.

COMMANDS AND TOOLS

systemctl list-dependencies <service>	Show full dependency tree
systemctl show <service>	Show detailed unit properties
systemd-analyze critical-chain	Show the boot critical chain
systemd-analyze verify <unit-file>	Verify unit file syntax and issues
journalctl -b	View boot logs for unit failures
systemctl is-active <service>	Check if service is active
systemctl is-enabled <service>	Check if service is enabled

TYPICAL SERVICE STARTUP ORDERING (EXAMPLE)



Flow: Network ready + Storage mounted → Database → Application → Reverse proxy

If any required unit fails, dependent units will not start.

EXAMPLES

```
# systemctl list-dependencies --tree myapp.service
myapp.service
├─● network-online.target
│ └─● NetworkManager-wait-online.service
│   └─● NetworkManager.service
└─● mysql.service
  └─● mariadb.service
    └─● basic.target
      └─● sockets.target
● = Required  ○ = Wanted  ◉ = Up/Active
```

```
# systemctl show myapp.service | grep -E 'After/Requires/Wants'
After=network-online.target mysql.service
Requires=mysql.service
Wants=network-online.target
Before=nginx.service
Description=My Application Service
```

```
# systemd-analyze critical-chain
graphical.target @12.930s
└─multi-user.target @12.930s
  └─myapp.service @12.531s +398ms
    └─mysql.service @8.243s +4.287s
      └─network-online.target @7.102s +1.140s
        └─NetworkManager-wait-online.service @6.901s +201ms

~ 12.930s (boot time)
```

```
# journalctl -b -u myapp.service
-- Logs begin at Mon 2025-07-22 00:00:00
-- myapp.service - My Application Service
Jul 22 13:01:08 server systemd[1]:
Starting My Application Service...
Jul 22 13:01:08 server myapp[1234]:
Application starting..
Jul 22 13:01:09 server myapp[1234]:
Application started successfully.
Jul 22 13:01:09 server systemd[1]:
Started My Application Service.
```

WHY ORDERING DOES NOT CREATE A DEPENDENCY

- After= only controls start sequence, not requirement.
- If the unit after= fails, this unit can still start.
- Use Requires= (strong) or Wants= (weak) for dependency.
- Example:

```
[Unit]
After=network-online.target
# No Requires or Wants
# myapp.service can start even if network is down
```

DEPENDENCY CYCLES (A COMMON PROBLEM)

Cycle example:

A.service After=B.service
B.service After=A.service

Impact:

- systemd cannot determine start order.
- One or both services may timeout or fail.
- Boot may hang or enter degraded state.



How to fix:

- ✓ Remove one of the After=.
- ✓ Use Wants/Requires instead.
- ✓ Break the cycle with a third unit.

DESIGNING A SERVICE THAT WAITS FOR READINESS

Use network and storage targets:

```
[Unit]
Description=My App Service
After=network-online.target remote-fs.target
Requires=network-online.target
Wants=network-online.target
After=mysql.service
Requires=mysql.service
```

- network-online.target → Waits until network is fully up.
- remote-fs.target → Waits for network filesystems.
- database service → Ensures DB is available first.

SERVICE DEPENDENCY WORKFLOW



BEST PRACTICES

- ✓ Always declare both ordering (After/) and dependency (Requires/Wants).
- ✓ Prefer network-online.target over network.target for network readiness.
- ✓ Use systemd-analyze critical-chain to understand boot delays.
- ✓ Validate unit files before deployment with systemd-analyze verify.
- ✓ Keep dependencies minimal and well documented.

COMMON MISTAKES

- ✗ Using only After= without Requires= or Wants=.
- ✗ Creating dependency cycles.
- ✗ Forgetting to reload systemd after modifying unit files.
- ✗ Assuming the service is enabled when it is only started manually.
- ✗ Not checking logs when a dependent service fails.



VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

SERVICE RESTART AND FAILURE CONTROL

Think Like a Senior Production Engineer

QUESTIONS 231-235

231 How do systemd restart policies work ?

232 When should Restart=on-failure be used instead of Restart=always ?

233 What causes the "start request repeated too quickly" error ?

234 How do StartLimitIntervalSec and StartLimitBurst prevent restart storms ?

235 How do you stop a failing service from creating an endless recovery loop ?

SYSTEMD RESTART POLICIES (Restart=)

POLICY	DESCRIPTION	WHEN IT RESTARTS
no	Do not restart the service.	Never
on-success	Restart only if the service exits successfully (exit code is success).	Exit status is success (0 by default)
on-failure	Restart only if the service fails (non-zero exit status, signal, timeout).	Exit status is failure / timeout / unclean termination
on-abnormal	Restart on abnormal termination (signal, timeout) but not on clean exit with failure code.	Signal, timeout, core dump, etc.
on-abort	Restart if the service aborts (e.g. via SIGABRT).	SIGABRT / abort signal
always	Always restart no matter how the service exits.	Always (be careful in production)

IMPORTANT DIRECTIVES

DIRECTIVE	TYPE	PURPOSE	EXAMPLE
Restart=	Behavior	Defines when systemd restarts the service.	Restart=on-failure
RestartSec=	Timing	Delay before restarting the service.	RestartSec=5s
StartLimitIntervalSec=	Rate Limit	Time window to track restart attempts.	StartLimitIntervalSec=60s
StartLimitBurst=	Rate Limit	Maximum number of restarts allowed within the interval.	StartLimitBurst=3
TimeoutStartSec=	Timeout	Time to wait for the service to start.	TimeoutStartSec=30s
TimeoutStopSec=	Timeout	Time to wait for the service to stop.	TimeoutStopSec=30s
SuccessExitStatus=	Behavior	Exit codes considered as "success" (so restart is not triggered).	SuccessExitStatus=0 143

CAUSE: "START REQUEST REPEATED TOO QUICKLY"

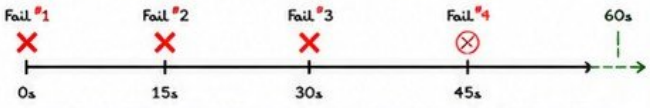
Service Fails to Start → systemd Attempts Restart → Exceeds Limit → Start request repeated too quickly

Common reasons:

- Service crashes repeatedly on startup
- Service takes too long to start (TimeoutStartSec hit)
- Wrong configuration or missing dependencies
- System resource exhaustion (CPU, memory, disk, ports)

RATE LIMITING: HOW IT PREVENTS RESTART STORMS

StartLimitIntervalSec = 60s - StartLimitBurst = 3



Within 60 seconds, 3 failures reached → Further starts are blocked.
systemd will not attempt again until the interval has passed.

STOPPING A FAILING SERVICE FROM AN ENDLESS RECOVERY LOOP

- ✓ Use Restart=on-failure instead of Restart=always.
- ✓ Set an appropriate RestartSec= delay.
- ✓ Configure StartLimitIntervalSec= and StartLimitBurst=.
- ✓ Fix the root cause (config, dependencies, resources).
- ✓ Use SuccessExitStatus= to treat expected codes as success.
- ✓ Temporarily disable restarts during investigation:

```
# systemctl edit myapp.service
[Service]
Restart=no
```

→ Override restart behavior

PRODUCTION WORKFLOW: SYSTEMD FAILURE AND RESTART LOGIC

1. Service Fails — Process exits, crashes, times out, or is killed.
2. systemd Records Exit Status — Exit code, signal, and result are stored.
3. Restart Policy Evaluated — systemd checks Restart= directive.
4. Rate Limit Checked — StartLimitIntervalSec and StartLimitBurst are evaluated.
5. Restart or Failed State — If allowed → Restart. If limit reached → enter failed state.

INSPECT AND DIAGNOSE

```
# Check service status and result
systemctl status myapp.service

# View recent logs
journalctl -u myapp.service -xe

# Show how many restarts happened
systemctl show myapp.service -p NRestarts

# Check rate limit configuration
systemctl show myapp.service -p StartLimit*
```

COMMON FIXES

- ✓ Correct configuration errors
- ✓ Ensure required dependencies are available
- ✓ Increase timeouts if service needs more time to start
- ✓ Fix crashes or memory issues
- ✓ Adjust restart and rate limit values carefully

EXAMPLE UNIT SNIPPET

```
[Unit]
Description=MyApp Service
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
ExecStart=/usr/bin/myapp
Restart=on-failure
RestartSec=5s
StartLimitIntervalSec=60s
StartLimitBurst=3
TimeoutStartSec=30s
TimeoutStopSec=30s
SuccessExitStatus=0 143

[Install]
WantedBy=multi-user.target
```

BEST PRACTICES

- ✓ Prefer on-failure for most services.
- ✓ Always set rate limits.
- ✓ Use meaningful timeouts.
- ✓ Monitor restarts and failures.
- ✓ Investigate root cause, don't just change settings.
- ✓ Document expected exit codes with SuccessExitStatus.

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

RESOURCE LIMITS AND ULIMIT

QUESTIONS 236-240

236 What resources can Linux limit for a process or user ?

237 What is the difference between soft and hard limits ?

238 How can file descriptor exhaustion cause a production outage ?

239 How do PAM limits, shell limits, and systemd limits interact ?

240 How do you apply resource limits to a production service ?

RESOURCES THAT CAN BE LIMITED

- **Open files** (file descriptors) — Maximum number of files/sockets a process can open.
- **Processes** (nproc) — Maximum number of processes/threads a user can start.
- **Locked memory** (memlock) — Memory that can be locked into RAM.
- **Core file size** (core) — Maximum size of core dump files.
- **Stack size** (stack) — Maximum size of the process stack.
- **CPU time** (cpu) — Maximum CPU time (in seconds) the process can use.
- **Address space** (as) — Maximum virtual memory (address space) a process can use.

SOFT LIMIT vs HARD LIMIT

SOFT LIMIT (rlim_cur)	HARD LIMIT (rlim_max)
✓ The current limit.	✓ The maximum ceiling.
✓ Can be increased up to the hard limit.	✓ Can only be increased by root.
✓ Applies to the running process.	✓ Cannot be exceeded.

Example (open files):
Soft limit = 1024
Hard limit = 4096
You can raise soft to 4096, but never above hard.

HOW LIMITS ARE APPLIED (ORDER OF PRECEDENCE)

1. **Kernel Default Limits** — Built-in defaults for the system.
2. **PAM Limits** (/etc/security/limits.conf and limits.d/) — Applied at user login.
3. **Shell Limits** (ulimit in shell profile) — Affects the current shell session only.
4. **systemd Limits** (Service unit file) — Highest priority for services to override others.

Effective limit = the most restrictive value from all applicable layers.

COMMON COMMANDS AND FILES

COMMAND	PURPOSE	EXAMPLE
ulimit -a	Show current shell limits	ulimit -a
ulimit -n 8192	Set soft limit for open files in current shell	ulimit -n 8192
prlimit --pid <PID>	Show or set limits for running process	prlimit --pid 1234
prlimit --pid <PID> --nofile=8192:8192	Set soft:hard for open files	prlimit --pid 1234 --nofile=8192:8192
cat /proc/<PID>/limits	View limits of a running process	cat /proc/1234/limits

IMPORTANT FILES

- /etc/security/limits.conf — Main PAM limits file
- /etc/security/limits.d/ — Drop-in directory for additional limits
- /etc/systemd/system/*.service — Service unit files with Limit* directives
- /proc/<PID>/limits — Actual limits applied to a running process

APPLYING LIMITS TO A PRODUCTION SERVICE (systemd)

Edit the service unit file

```
[Service]
# Limit the number of open files
LimitNOFILE=8192

# Limit the number of processes/threads
LimitNPROC=4096

# Lock up to 256MB of memory
LimitMEMLOCK=268435456

# CPU time limit (in seconds)
LimitCPU=1h
# Address space limit (virtual memory)
LimitAS=8G
```

Steps

1. Edit the unit file.
2. Add or update Limit* directives.
3. Run: systemctl daemon-reload
4. Restart the service.
5. Verify applied limits.

COMMON LIMIT DIRECTIVES IN systemd

DIRECTIVE	LIMIT TYPE	EXAMPLE
LimitNOFILE=	Max open files	LimitNOFILE=8192
LimitNPROC=	Max processes	LimitNPROC=4096
LimitMEMLOCK=	Max locked memory	LimitMEMLOCK=256M
LimitCORE=	Max core file size	LimitCORE=0
LimitSTACK=	Max stack size	LimitSTACK=16M
LimitCPU=	CPU time limit	LimitCPU=1h
LimitAS=	Max address space	LimitAS=8G

COMMAND EXAMPLES

```
# 1. View all shell limits
$ ulimit -a
...
open files (-n)          1024
max user processes (-u)  4096
...
```

```
# 2. Show running process limits
$ cat /proc/1234/limits
Limit                     Soft   Hard
Max open files            1024   4096
Max processes             4096   4096
Max locked memory         65536  65536
...
```

```
# 3. Show limit for a process using prlimit
$ prlimit --pid 1234
RESOURCE
NOFILE     1024:4096
NPROC      4096:4096
MEMLOCK    65536:65536
...
```

```
# 4. Change limit for a process
$ prlimit --pid 1234 --nofile=8192:8192
$ prlimit --pid 1234 --nproc=8192:8192
# (requires root)
```

PRODUCTION SCENARIO

A web service returns "too many open files" even though CPU and memory are healthy.

What happened?

- The process hit its open file limit (ulimit -n).
- New connections cannot be accepted.
- Service becomes unresponsive.
- Error logs show: "Too many open files".

INVESTIGATION WORKFLOW

1. Confirm the error in app/logs.
2. Check open files limit: ulimit -n (or cat /proc/<PID>/limits)
3. Check current open files count: lsof -p <PID> | wc -l
4. Increase the limit (temporarily): ulimit -n 8192 or use prlimit
5. Update systemd unit with LimitNOFILE.
6. Reload and restart service.
7. Monitor to ensure stability.

BEST PRACTICES

- ✓ Set limits based on workload, not guesses.
- ✓ Keep hard limit reasonably above soft limit.
- ✓ Use systemd limits for services (most reliable).
- ✓ Document limits for every production service.
- ✓ Monitor open files and process counts.
- ✓ Alert before limits are reached.

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

CGROUPS AND RESOURCE GOVERNANCE

QUESTIONS 241-245

241 What problem do Linux control groups solve ?

242 How do cgroups control CPU, memory and process allocation ?

243 What is the difference between cgroups v1 and v2 ?

244 How does systemd organize services into cgroups ?

245 How do containers use cgroups to enforce resource limits ?

WHAT PROBLEM DO CGROUPS SOLVE?

cgroups (control groups) solve the problem of resource contention and noisy neighbors.

Without cgroups:

- A single process or service can consume unlimited CPU, memory, or I/O.
- One application can affect the performance or availability of others.
- System resources cannot be fairly shared or isolated.

With cgroups:

- ✓ Resources are allocated, limited and monitored per group.
- ✓ Services get predictable performance.
- ✓ The system remains stable under load.
- ✓ Isolation and multi-tenancy are possible.

RESOURCE CONTROLLERS

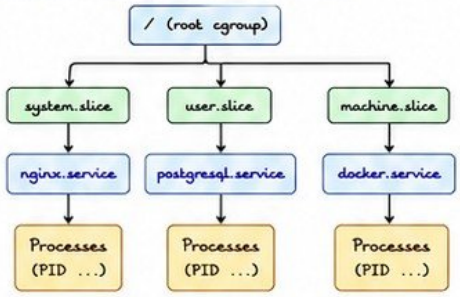
CONTROLLER	PURPOSE
CPU	Limits CPU time and shares among groups.
Memory	Limits and accounts for memory usage.
I/O	Limits disk I/O throughput and IOPS.
PIDs	Limits the number of processes and threads.
Cpuset	Restricts which CPUs and NUMA nodes a group can use.

CGROUPS v1 vs CGROUPS v2

FEATURE	CGROUPS v1	CGROUPS v2
Hierarchy	Multiple hierarchies (one per controller)	Unified hierarchy (single tree)
Controllers	Enabled per hierarchy	All controllers in one place
Management	Complex (multiple mounts)	Simpler (single mount point)
Delegation	Limited and error-prone	Improved and standard
Performance	Higher overhead	Lower overhead
Features	Some controllers missing (e.g., io, pids incomplete)	All major controllers integrated
Mount point	/sys/fs/cgroup/<controller>	/sys/fs/cgroup

HOW SYSTEMD ORGANIZES SERVICES INTO CGROUPS

- systemd uses cgroups to isolate and manage every service.
- All processes of a service are placed in the service's cgroup.
- systemd creates a hierarchy of slices and scopes.
- This allows resource accounting, limits, monitoring and control.



HOW CONTAINERS USE CGROUPS

Containers rely on cgroups for isolation and resource governance.

Mechanism:

1. The container runtime (e.g., Docker, containerd, CRI-O) creates a cgroup for the container.
2. Resource limits in the container configuration are applied to that cgroup.
3. All container processes run inside that cgroup.
4. The kernel enforces the limits set by the runtime.

Result:
Containers cannot exceed the limits defined for CPU, memory, I/O, PIDs or other resources.

COMMANDS AND TOOLS

COMMAND	DESCRIPTION	EXAMPLE
systemd-cgls	Display the cgroup hierarchy in a tree.	systemd-cgls --no-pager
systemd-cgtop	Real-time monitoring of cgroup resource usage.	systemd-cgtop
systemctl show	Show properties of a service including its cgroup path.	systemctl show nginx.service -p ControlGroup
cat /sys/fs/cgroup/cgroup.controllers	List available controllers in cgroups v2.	cat /sys/fs/cgroup/cgroup.controllers
systemd-run	Run a command in a transient cgroup.	systemd-run --scope -p MemoryMax=500M myapp
systemctl set-property	Set properties (limits) on a running service.	systemctl set-property nginx.service MemoryMax=512M
cat /proc/<PID>/cgroup	Show which cgroup the process belongs to.	cat /proc/1234/cgroup

CGROUP HIERARCHY VIEW (EXAMPLE OUTPUT)

```
$ systemd-cgls --no-pager
Control group /
├─›.slice
│ ├─ system.slice
│ │ ├─ nginx.service
│ │ │ ├─›-1234 nginx: master process
│ │ │ ├─› 1235 nginx: worker process
│ │ │ └─›-1236 nginx: worker process
│ │ ├─ sshd.service
│ │ │ └─›-1450 sshd: user@pts/0
│ ├─ user.slice
│ │ ├─ user-1000.slice
│ │ │ ├─›-session-2.scope
│ │ │ │ └─›-1540 bash
└─›machine.slice
  ├docker.service
  ├─docker-<id>.scope
  │ └─›-2345 containerd-shim
```

RESOURCE GOVERNANCE WORKFLOW

1. Define resource requirements (CPU, memory, I/O, PIDs, etc.)
2. Create or configure cgroup/ service with limits
3. Attach processes to the cgroup (systemd or container runtime)
4. Kernel enforces limits using controllers
5. Monitor usage and adjust as needed

BEST PRACTICES

- ✓ Set appropriate limits for all critical services.
- ✓ Monitor cgroup usage regularly.
- ✓ Prevent noisy neighbor issues with limits.
- ✓ Use slices for logical grouping.
- ✓ Test under load to validate limits.
- ✓ For containers, always set resource limits.
- ✓ Document cgroup policies and thresholds.

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer



CPU SCHEDULING AND PROCESS PRIORITY

QUESTIONS 246-250

246. How does the Linux scheduler decide which process runs next ?
247. What is the difference between nice value and real-time priority ?
248. When should nice and renice be used in production ?
249. How do CPU affinity and cpusets affect performance ?
250. What risks come with real-time scheduling policies ?

KEY CONCEPTS

CFS (Completely Fair Scheduler)
The default scheduler for normal processes. It tries to give each runnable task a fair share of CPU time.

Nice Values
A user-space hint (-20 to 19) that affects the weight of a process in the CFS run queue. Lower value = higher priority.

Scheduler Classes
- Normal (CFS) - for standard processes
- Batch - non-interactive batch jobs
- Idle - only runs when system is idle
- Real-time (FIFO, RR, Deadline) - for time critical tasks

Run Queue
Per-CPU list of runnable tasks waiting for CPU time.

Time Slices
Amount of CPU time a task gets before the scheduler may switch to another task.

Real-time Scheduling
Preempts normal tasks. Used for strict latency or timing guarantees.

NICE VALUE vs REAL-TIME PRIORITY

NICE VALUE (Normal Scheduling)	REAL-TIME PRIORITY (Real-Time Scheduling)
• Range: -20 (highest) to 19 (lowest)	• Range: 1 (lowest) to 99 (highest)
• Used by CFS	• Uses policies: FIFO, RR, Deadline
• Influences share of CPU time	• Preempts normal tasks
• Cannot guarantee latency	• Must be used carefully
• Set by user or application	• Requires root privileges
• Example: nice -n -10 ./backup.sh	• Example: chrt -f -p 50 1234



Key Point:
Nice affects how much CPU you get.
Real-time priority affects when you run.
They are completely different mechanisms.

WHEN TO USE NICE AND RENICE

Use Cases
- Lower priority of CPU-intensive batch jobs
- Prevent one process from impacting interactive users
- Adjust priority without stopping the process
- Fine-tune performance during incidents

Examples
- Start a backup job with low priority:

```
nice -n 10 ./backup.sh
```

- Lower priority of a running process:

```
renice 10 -p 12345
```

CPU AFFINITY AND CPUSETS

CPU AFFINITY	CPUSETS (cgroups)
• Binds a process to specific CPU cores.	• Restricts a group of processes to a set of CPUs.
• Reduces cache misses and context switches.	• Also controls memory nodes.
• Useful for latency-sensitive or CPU-bound applications.	• Ensures workloads stay within defined boundaries.
• Set with taskset or sched_setaffinity().	• Used by containers and systemd slices.

Example

```
taskset -c 2,3 12345        # Run PID 12345 on CPU 2 and 3
# View current affinity
taskset -p 12345
```

SCHEDULER CLASSES OVERVIEW

Class	Policy	Description	Use Case
Normal	CFS	Default for most processes	Web, databaess, general apps
Batch	SCHED_BATCH	For non-interactive batch jobs	Cron jobs, batch processing
Idle	SCHED_IDLE	Runs only when system is idle	Very low priority background tasks
Real-Time	SCHED_FIFO	First-In-First-Out	Audio, control systems
Real-Time	SCHED_RR	Round-Robin	Real-time with time slices
Real-Time	SCHED_DEADLINE	Deadline based	Critical real-time workloads

COMMANDS AND TOOLS

COMMAND	PURPOSE	EXAMPLE
nice	Start a process with a nice value	nice -n 10 ./script.sh
renice	Change nice value of a running process	renice -n 5 -p 12345
chrt	Show or set real-time scheduling	chrt -f -p 50 12345
taskset	Set or view CPU affinity	taskset -c 0-3 ./app
ps -eo pid,ni,pri, rtprio,psr,stat,cmd	Show priority, CPU, and status	ps -eo pid,ni,pri, rtprio,psr,stat,cmd
schedtool	Query/modify scheduler settings	schedtool -p -a 12345

MONITORING CPU AND PRIORITY

```
# Show processes with priority and CPU information
ps -eo pid,ni,pri,rtprio,psr,stat,cmd | head -n 10
```

PID	NI	PRI	RTPRIO	PSR	STAT	CMD
2234	0	120	-	0	S	nginx: worker process
2345	10	130	-	1	R	backup.sh
3456	-5	100	-	2	S	java
4567	0	120	50	3	S	rt-task (FIFO)

Columns:
NI = nice value | PRI = kernel priority | RTPRIO = real-time priority
PSR = CPU core | STAT = process state | CMD = command

PRODUCTION WORKFLOW

1. Identify CPU-heavy process
2. Check current priority and policy
3. Check CPU affinity and cpuset
4. Assess CPU contention and load
5. Adjust priority or affinity safely
6. Monitor impact and fine-tune



RISKS OF REAL-TIME SCHEDULING

- Can starve normal processes
- System may become unresponsive
- Hard to debug timing issues
- Misconfiguration can cause outages
- Use only when strict latency is required





VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

LOAD AVERAGE AND RUN QUEUE ANALYSIS

QUESTIONS 251-255

251. What does Linux load average measure ?
252. Why is a load average of 8 acceptable on one server but critical on another ?
253. How do runnable and uninterruptible processes affect load average ?
254. How do you distinguish CPU saturation from storage-related load ?
255. Why can load average remain high while CPU usage appears low ?

WHAT DOES LINUX LOAD AVERAGE MEASURE?

Load average is the average number of processes in the run queue waiting for CPU time, including:

- Runnable (R) processes – waiting for CPU.
- Uninterruptible (D) processes – waiting for I/O (disk, network, etc.).

Key Point:
Load average does NOT measure CPU usage.
It measures demand for CPU + system resources.
Size of the run queue = system pressure.

UNDERSTANDING LOAD AVERAGE

Load Average (1, 5, 15 min) = Average number of Runnable (R) + Uninterruptible (D) processes → Run queue pressure on the system

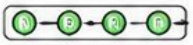
WHY LOAD AVERAGE = 8 MAY BE ACCEPTABLE OR CRITICAL

Server Type	CPU Cores	Load Avg = 8	Interpretation
Small server	(2 cores)	8 >> 2	Critical – heavy contention, high latency, slow response
Medium server	(8 cores)	8 ≈ 8	Acceptable – all cores busy but not overloaded
Large server	(32 cores)	8 << 32	Very light load – plenty of capacity available

HOW PROCESSES AFFECT LOAD AVERAGE

RUNNABLE PROCESSES (R)

- Ready to run.
- Waiting for CPU time.
- Increase load when CPU is busy.



Run Queue

UNINTERRUPTIBLE PROCESSES (D)

- Waiting for I/O to complete.
- Cannot be killed.
- Causes load to increase even if CPU is idle.



I/O Wait Queue

DISTINGUISHING CPU SATURATION FROM STORAGE-RELATED LOAD

Indicator	CPU Saturation	Storage-Related Load
Load Average	High	High
CPU Usage (top)	High (80–100%)	Low or Normal
%wa (vmstat)	Low (<5%)	High (>20%)
CPU Run Queue	Long	Short
Disk I/O (iostat)	Normal	High (await, svctm, util)
Processes in D state	Few	Many
Response Time	High (CPU bound)	High (I/O bound)

WHY LOAD AVERAGE CAN BE HIGH BUT CPU USAGE LOW

- Many processes are in uninterruptible sleep (D state).
- They wait on slow disk I/O, NFS, network, or locks.
- These processes stay in the run queue.
- CPU is idle, but processes cannot run.
- Result: High load average, low CPU usage.



Load Average High ↑

COMMANDS AND TOOLS

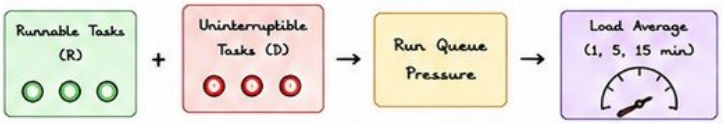
COMMAND	PURPOSE	EXAMPLE / NOTES
uptime	Show load average and uptime	uptime
w	Show users, load, and running processes	w
top	Real-time system overview	top
vmstat 1	Show CPU, memory, I/O, and run queue	vmstat 1
sar -q	Show run queue and load statistics	sar -q 1 5
ps -eo state,pid,cmd	List processes with state	ps -eo state,pid,cmd | grep -E "R|D"
cat /proc/loadavg	Show raw load average values	cat /proc/loadavg

EXAMPLE: INTERPRETATION

Example 1: CPU Bound
uptime: 3.10, 2.90, 2.70 (Server with 2 cores)
top: ≈ CPU usage 95%
vmstat: %wa 2%
Interpretation: High load due to CPU saturation.
Solution: optimize CPU usage, scale up.

Example 2: I/O Bound
uptime: 8.45, 7.90, 7.30 (Server with 8 cores)
top: ≈ CPU usage 10%
vmstat: %wa 45%
Interpretation: High load due to I/O wait.
Solution: fix storage, network, or I/O bottleneck.

VISUAL MODEL



BEST PRACTICES

- ✓ Always compare load average with number of CPU cores.
- ✓ Check %wa in vmstat – high %wa means I/O problem.
- ✓ Inspect D state processes during high load.
- ✓ Use sar and iostat for long-term trend analysis.
- ✓ Address root cause, not just symptoms.

COMMON MISTAKES

- ✗ Assuming high load average always means high CPU usage.
- ✗ Ignoring I/O wait and D state processes.
- ✗ Killing processes without understanding the root cause.
- ✗ Not monitoring run queue length over time.
- ✗ Not scaling storage when required.



VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

MEMORY PRESSURE AND LINUX MEMORY ACCOUNTING

QUESTIONS 256-260

256 How does Linux use free memory for buffers and cache ?

257 What is the difference between free, available, cached and used memory ?

258 How do you identify genuine memory pressure ?

259 What are anonymous memory, page cache and slab memory ?

260 How do you determine whether memory usage is caused by an application or the kernel ?

HOW LINUX USES FREE MEMORY

Linux uses free memory to improve performance.

- File data is stored in the page cache.
- Disk operations use buffers.
- Directory metadata and inodes are stored in slab cache.
- This cache acts as a fast memory layer.
- When applications need memory, Linux releases cache automatically.

Key Point:
Free memory that is not used is wasted memory. Linux will always try to use it for something useful.

FREE vs AVAILABLE vs CACHED vs USED

Metric	Meaning
Free	Completely unused memory that is not used for anything.
Available	Memory available for applications without swapping.
Cached	Memory used for page cache and slab cache. Can be reclaimed.
Used	Memory used by applications, kernel and cache.

Available memory is the most important metric.

HOW TO IDENTIFY GENUINE MEMORY PRESSURE

Signs of memory pressure:

1. Available memory keeps dropping consistently.
2. System starts using swap.
3. Swap in (si) increases in vmstat.
4. The system becomes slow or unresponsive.
5. The OOM Killer starts terminating processes.
6. kswapd is constantly active.

Important:
High used memory is normal.
High used + low available + swap usage is the real problem.

TYPES OF MEMORY IN LINUX

Anonymous Memory	Memory used by applications for data that is not backed by files. (Heap, stack, malloc, etc.)
Page Cache	Cached data from files to speed up disk reads. Can be freed when needed.
Slab Memory	Memory used by the kernel for internal data structures (inodes, dentries, sockets, etc.)
Buffers	Used for block device I/O operations.

APPLICATION vs KERNEL MEMORY USAGE

Application Memory
- Anonymous pages
- Mapped files
- User space allocations
- Shown in: smem, ps, pmap

VS

Kernel Memory
- Page cache
- Slab cache
- Kernel structures
- Shown in: /proc/meminfo, slabtop, vmstat

Check with:

```
smem -r
ps -eo pid,%mem,rss,vsz,cmd
pmap -x <PID>
```

Check with:

```
cat /proc/meminfo
slabtop
vmstat 1
```

SYSTEM MEMORY ACCOUNTING MODEL

Applications (Anonymous Memory) + Page Cache (File Cache) + Kernel Memory (Slab) + Buffers + Swap (When used) = System Memory Usage

- Linux will reclaim Page Cache and Buffers before using Swap.
- Slab memory can grow and shrink based on kernel needs.
- Swap is used only when necessary.

Key Point:
High memory usage is not a problem if Available memory is healthy and Swap is not being used.

COMMANDS AND TOOLS

COMMAND	PURPOSE	EXAMPLE / NOTES
free -h	Show summary of memory usage.	Shows total, used, free, shared, buff/cache, available.
vmstat 1	Real-time memory and system stats.	Watch si, so, free, buff, cache, inact.
cat /proc/meminfo	Detailed memory information.	Complete breakdown of all memory metrics.
smem	Proportional Set Size for processes.	Shows which processes use the most memory.
slabtop	Real-time slab memory usage.	Top kernel caches consuming memory.
pmap -x <PID>	Detailed memory map of a process.	Shows RSS, private, shared, dirty, mapped regions.
ps -eo pid,%mem,rss,vsz,cmd	List processes by memory usage.	Sort by %MEM to find top memory consumers.
sar -r 1	Historical memory statistics.	Requires sysstat. Good for trend analysis.

EXAMPLE OUTPUTS

1. free -h

```
              total        used        free      shared  buff/cache   available
Mem:           15Gi        11Gi       1.2Gi       300Mi       2.8Gi       2.9Gi
Swap:         4.0Gi       512Mi       3.5Gi
```

2. vmstat 1 (excerpt)

```
procs -----------memory---------- ---swap-- -----io---- -system-- -----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 2  1 524288 130000 250000 1800000    0    0    30    80  120  200  5  3 90  1  0
```

3. cat /proc/loadavg

```
2.34 1.89 1.45 3/789 12345
```

TROUBLESHOOTING MEMORY PRESSURE

1. Check available memory: free -h
2. Check swap usage: vmstat 1
3. Identify top processes: smem -r
4. Check kernel usage: slabtop
5. Check recent OOM events: dmesg | grep -i oom
6. Tune or scale based on findings.

BEST PRACTICES

- ✓ Monitor available memory, not just used.
- ✓ Set alerts on swap usage and OOM events.
- ✓ Tune applications for memory efficiency.
- ✓ Use memory limits for containers and services.
- ✓ Investigate before adding more RAM.

COMMON CAUSES OF HIGH MEMORY USAGE

- Memory leaks in applications
- Large file caches (normal)
- Excessive slab cache growth
- Too many running processes
- Misconfigured services or containers

COMMON MISTAKES

- ✗ Reacting to high used memory.
- ✗ Ignoring swap usage.
- ✗ Killing random processes.
- ✗ Not checking for memory leaks.
- ✗ Over-provisioning memory blindly.

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer



OOM KILLER AND MEMORY FAILURE INVESTIGATION

QUESTIONS 261-265

261 What triggers the Linux OOM Killer ?

262 How does Linux decide which process to terminate ?

263 What do oom_score and oom_score_adj control ?

264 How do you prove that an application was killed by the OOM Killer ?

265 How do you prevent recurring OOM incidents ?

WHAT TRIGGERS THE OOM KILLER?

- The OOM Killer is triggered when the kernel cannot allocate memory for a process.
- This happens after reclaiming memory and using swap, yet memory is still unavailable.
- Common scenarios:
  - Physical RAM is exhausted
  - Swap is exhausted
  - Memory cgroup limit (memory.max) is hit
  - Kernel needs memory urgently (e.g., for critical allocations)

HOW LINUX SELECTS A VICTIM

The OOM Killer assigns a badness score to each process. Higher score = more likely to die.

Factors considered:

- ✓ Memory usage (RSS + cache)
- ✓ oom_score_adj (adjustment)
- ✓ Nice value (lower priority if higher nice)
- ✓ Capabilities and privileges
- ✓ Whether the process is essential (root/special services get lower score)

OOM SCORE AND OOM SCORE ADJ

oom_score (/proc/<PID>/oom_score)

- Dynamic score from 0 to 1000.
- 0 = least likely to be killed.
- 1000 = most likely to be killed.
- Calculated by kernel based on memory usage.

oom_score_adj (/proc/<PID>/oom_score_adj)

- Adjustment from -1000 to +1000.
- -1000 = never kill this process.
- 0 = neutral (default).
- +1000 = always kill this process first.

HOW TO PROVE A PROCESS WAS KILLED BY OOM

- Kernel logs contain "Killed process" message.
- dmesg or journalctl -k will show OOM details.
- The log includes:
  - Process name and PID
  - Memory usage
  - Total system memory
  - The selected victim
  - Reason for kill

Key Point:
If you see "Out of memory: Kill process", your application was killed by the OOM Killer.

PREVENTING RECURRING OOM INCIDENTS

- ✓ Add more RAM if required.
- ✓ Reduce application memory usage.
- ✓ Set memory limits using cgroups/systemd (MemoryMax=).
- ✓ Use swap as a safety net (not as a primary fix).
- ✓ Tune caches: vm.swappiness, dirty_ratio.
- ✓ Exclude critical services from OOM kills (oom_score_adj=-1000).
- ✓ Monitor memory and set alerts early.

COMMANDS AND EVIDENCE

COMMAND	PURPOSE	EXAMPLE / NOTES
dmesg -T | grep -i oom	Show OOM events in kernel ring buffer	dmesg -T | grep -i oom
journalctl -k	View kernel logs (includes OOM events)	journalctl -k -b
cat /proc/<PID>/oom_score	Show current OOM score (0-1000)	cat /proc/1234/oom_score
cat /proc/<PID>/oom_score_adj	Show OOM score adjustment (-1000 to +1000)	cat /proc/1234/oom_score_adj
systemctl show <service>	Show service resource limits and settings	systemctl show myapp.service | grep -i memory
memory.max (cgroup v2)	Hard memory limit for a cgroup/service	cat /sys/fs/cgroup/myapp.service/memory.max
MemoryMax= (systemd)	Limit memory usage of a service	MemoryMax=1G in unit file

SYSTEMD MEMORY LIMIT EXAMPLE

```
# /etc/systemd/system/myapp.service
[Service]
ExecStart=/usr/bin/myapp
User=myapp
MemoryMax=1G
MemoryHigh=800M
MemorySwapMax=256M
OOMScoreAdjust=-1000
Restart=on-failure
```

MemoryMax=1G means the service will be killed if it tries to use more than 1 GB of RAM.

OOMScoreAdjust Values
- -1000 = Never kill
- -500 = Very unlikely
- 0 = Default
- +500 = More likely
- +1000 = Kill first

Important
Setting OOMScoreAdjust=-1000 protects critical services from being killed during memory crisis.

PRODUCTION WORKFLOW: OOM KILLER SEQUENCE

1. Memory Pressure — Applications consume memory. Free memory starts dropping.
2. Reclaim Attempts — Kernel reclaims page cache, slab memory and buffers to free memory.
3. Swap Activity — Kernel starts using swap space to free physical memory.
4. Allocation Failure — A process requests memory but the kernel cannot allocate it.
5. OOM Selection — Kernel calculates scores and selects the process with the highest score.
6. Process Termination — The selected process is killed. Memory is freed immediately.
7. Recovery — Freed memory is used. System stabilizes and continues running.

EVIDENCE EXAMPLE (dmesg output)

```
[ 12345.678901] Out of memory: Kill process 1234 (myapp) score 987 or sacrifice child
[ 12345.678903] Killed process 1234 (myapp) total-vm:2048000kB, anon-rss:1500000kB
[ 12345.678905] oom_reaper: reaped process 1234 (myapp), now anon-rss:0kB, file-rss:0kB
```

BEST PRACTICES

- ✓ Monitor memory usage and trends.
- ✓ Set memory limits for services and containers.
- ✓ Protect critical services with OOMScoreAdjust.
- ✓ Test OOM scenarios in staging.
- ✓ Keep headroom for kernel and system daemons.

COMMON MISTAKES

- ✗ Assuming swap solves all memory problems.
- ✗ Ignoring early warning signs.
- ✗ Not setting memory limits for services.
- ✗ Killing the wrong process manually.
- ✗ Running without monitoring and alerts.

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

CPU BOTTLENECK INVESTIGATION

QUESTIONS 266-270

266 How do you confirm that CPU is the actual bottleneck ?

227 What is the difference between user, system, idle, iowait and steal time ?

268 How do you identify a single process or thread consuming CPU ?

269 How do you investigate CPU saturation inside a virtual machine ?

270 When is adding more CPU the wrong solution ?

CONFIRMING CPU IS THE BOTTLENECK

1. High CPU utilization sustained over time.
2. Load average is >= number of CPU cores.
3. Run queue length is consistently high.
4. CPU is busy even when I/O is healthy.
5. Adding I/O or memory does not improve performance.
6. Response time increases with CPU usage.

Key Point:
If CPU is not the bottleneck, increasing CPU will not improve performance.

CPU TIME BREAKDOWN

METRIC	MEANING	HIGH VALUE INDICATES
user	Time spent running application code in user space.	CPU busy executing application logic.
system	Time spent in kernel mode.	Kernel activity, system calls, context switches.
idle	CPU is idle, nothing to run.	Good. CPU is available.
iowait	CPU idle waiting for I/O to complete.	Storage or I/O subsystem is slow.
steal	CPU taken by the hypervisor.	Noisy neighbor or CPU contention in VM.

IDENTIFYING CPU-HOGGING PROCESS AND THREAD

Step 1: Find Top Processes

top or htop

Sort by %CPU. Find the process using the most CPU.

Step 2: Drill Down to Threads

```
ps -eo pid,tid,psr,%cpu,stat,cmd --sort=-%cpu | head
```

Shows threads (TID) sorted by CPU usage.

Step 3: Real-time Per-thread CPU

```
pidstat -u -t 1
```

Update every 1 second. Press Ctrl+C to stop.

What to Look For

- Single thread using 100% CPU.
- Many threads all using high CPU.
- threads bouncing across CPUs.
- Processes in D (uninterruptible sleep) usually indicate I/O issues, not CPU.

High CPU from one thread usually means a bug, loop, or heavy computation.

INVESTIGATING CPU SATURATION IN A VIRTUAL MACHINE

1. Check steal time:
   mpstat -P ALL 1
   High %steal => hypervisor contention.
2. Check CPU allocation to the VM:
   Verify vCPU count and CPU limits.
3. Check noisy neighbors on the host.
4. Check oversubscription on the host.
5. Check CPU ready time (if available):
   vmstat 1
   High r (run queue) inside VM.
6. Coordinate with infrastructure team for host-level metrics.

Key Point:
High steal time means the hypervisor, not your application, is the bottleneck.

WHEN ADDING MORE CPU IS THE WRONG SOLUTION

- ✗ Application has a CPU inefficiency (tight loop, bad algorithm).
- ✗ Single-threaded application cannot use more CPU.
- ✗ Lock contention or synchronization issues.
- ✗ External bottleneck (database, API, storage, network).
- ✗ Poor query performance or slow dependencies.
- ✗ I/O bound workload misdiagnosed as CPU bound.

Fix the root cause, not the symptom.

COMMANDS AND TOOLS REFERENCE

COMMAND	PURPOSE	EXAMPLE / NOTES
top	Real-time CPU overview	Press 1 to view per-CPU
htop	Interactive process viewer	F6 to sort by %CPU
mpstat -P ALL 1	Per-CPU utilization	Repeats every 1 sec
pidstat -u -t 1	Per-thread CPU usage	Press Ctrl+C to stop
ps -eo pid,tid,psr,%cpu,stat,cmd	Show threads with CPU and CPU core	--sort=-%cpu | head
perf top	CPU profiling (kernel + user)	Requires perf tools
sar -u 1	Historical CPU usage	From sysstat package
vmstat 1	System performance summary	Look at r, us, sy, id, wa, st

PRODUCTION WORKFLOW

1. Confirm Saturation — Verify CPU usage is sustained high and load is high.
2. Identify Affected CPUs — Use mpstat to find which CPUs are fully utilized.
3. Find Process and Thread — Use top/ps/pidstat to identify the exact process and thread.
4. Correlate with Workload — Understand what the application was doing at that time.
5. Profile and Analyze — Use perf top or other tools to identify the hot code path.
6. Correct Root Cause — Optimize code, fix query, adjust configuration, or resolve contention.

EXAMPLE: IDENTIFY CPU-HOGGING THREAD

```
$ ps -eo pid,tid,psr,%cpu,stat,cmd --sort=-%cpu | head
  PID   TID PSR %CPU STAT CMD
 4321  4321   3 98.7 R    java -Xmx2g -jar app.jar
 4321  4325   3 97.9 R    java -Xmx2g -jar app.jar
 4321  4326   2  1.2 S    java -Xmx2g -jar app.jar
 1245  1245   1  0.5 S    nginx: worker process
  987   987   0  0.3 S    systemd-journald
    1     1   0  0.0 S    systemd --system
```

Thread 4321/4321 is consuming ~99% CPU on core 3.

BEST PRACTICES

- ✓ Monitor CPU usage and load continuously.
- ✓ Alert on sustained high CPU and load.
- ✓ Analyze before scaling.
- ✓ Profile applications in staging first.
- ✓ Use performance baselines.
- ✓ Document and track CPU hotspots.

COMMON MISTAKES

- ✗ Adding more CPU without analysis.
- ✗ Ignoring steal time in virtualized environments.
- ✗ Focusing on process, not thread.
- ✗ Not correlating CPU spikes with workload.
- ✗ Blaming CPU when it is an I/O or lock issue.

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

CONTEXT SWITCHING AND SCHEDULER PRESSURE

QUESTIONS 271-275

271 What is a context switch ?

272 What is the difference between voluntary and involuntary context switching ?

273 When do high context-switch rates become a performance problem ?

274 How can excessive threads reduce application performance ?

275 How do you investigate scheduler pressure ?

WHAT IS A CONTEXT SWITCH?

- A context switch occurs when the CPU stops executing one thread/process and starts executing another.
- The scheduler saves the state of the current task and restores the state of the next task.
- It allows the kernel to share the CPU among many tasks.

Key Point:
Context switching is necessary, but too many switches waste CPU time and reduce throughput.

VOLUNTARY vs INVOLUNTARY CONTEXT SWITCH

Voluntary (Non-preemptive)

- The task gives up the CPU on its own.
- Examples:
  - Waiting for I/O
  - Sleeping
  - Blocking on a lock
  - Yielding CPU

Involuntary (Preemptive)

- The task is forced to give up the CPU.
- Caused by:
  - Timer interrupt
  - Higher priority task
  - CPU time slice expiry
  - Scheduler decision

WHEN DO HIGH CONTEXT SWITCH RATES BECOME A PROBLEM?

- Very high context switches per second (tens of thousands or more).
- CPU usage is high, but throughput is low.
- System feels "busy" but not doing useful work.
- Run queue length is consistently greater than the number of CPUs.
- High system time in top (sy).
- Applications experience latency and slow response.

Rule of Thumb:
If context switches per second are more than processes can efficiently handle, CPU time is wasted on switching instead of doing work.

HOW EXCESSIVE THREADS HURT PERFORMANCE

- More threads = more competition for CPU.
- Leads to more context switches, cache misses, and memory pressure.
- Threads spend more time waiting than working.
- CPU time is wasted on saving/restoring state.
- Lock contention increases.
- Diminishing returns: after a point, adding more threads reduces performance.

Key Point:
More threads do not always mean more performance. The goal is the right number of threads, not the most threads.

INVESTIGATING SCHEDULER PRESSURE

1. Check context switch rate.
2. Analyze run queue length.
3. Identify CPU-heavy processes and threads.
4. Check CPU utilization (user, system, iowait).
5. Check for lock contention and I/O waits.
6. Correlate with application workload.

COMMANDS AND TOOLS

COMMAND	PURPOSE	EXAMPLE / NOTES
`vmstat 1`	System performance report including context switches.	Look at: cs (context switches/sec), r (run queue), b (blocked).
`pidstat -w 1`	Per-process context switching (voluntary and involuntary).	Columns: cswch/s (voluntary), nvcswch/s (involuntary).
`sar -w`	Historical context switch statistics.	Shows context switches per second over time.
`perf stat`	Hardware performance counters.	`perf stat -e context-switches,cpu-migrations <command>`
`/proc/<PID>/status`	Per-process statistics.	`Voluntary_ctxt_switches` `Nonvoluntary_ctxt_switches`
`/proc/schedstat`	Scheduler statistics.	Shows time spent running, waiting, and timeslices per CPU.

EXAMPLE: CHECK CONTEXT SWITCH RATE

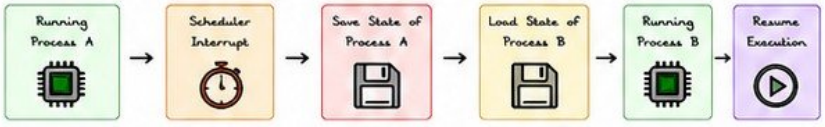
1. `vmstat 1` (Look at cs column)

```
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 4  1      0 512000 102400 204800    0    0   120    80 1500 2400 25 15 60  3  2
 6  2      0 412000  98000 210000    0    0   200   100 1600 4500 35 20 50  3  2
```

2. `pidstat -w 1` (Per-process)

```
Linux 5.15.0      (server)      05/22/2025      _x86_64_      (4 CPU)
12:00:01 PM       UID       PID   cswch/s   nvcswch/s   Command
12:00:01 PM         0       123     12.00      450.00   java
12:00:01 PM         0       456      5.00       30.00   nginx
12:00:01 PM         0       789      1.00        5.00   sshd
```

CONTEXT SWITCHING DIAGRAM (HIGH LEVEL)



This entire cycle is a context switch. Too many of these waste CPU time.

WHAT /PROC/SCHEDSTAT SHOWS (EXAMPLE)

```
# cat /proc/schedstat
cpu0  12345678  987654  543210  12345  67890
cpu1  23456789  876543  654321  22345  77890
cpu2  34567890  766432  754322  32345  87890
cpu3  45678901  654321  876543  42345  97890
```

Fields:

- `run_time` - Nanoseconds spent running tasks
- `runqueue_time` - Nanoseconds tasks waited in run queue
- `timeslices` - Number of timeslices
- `nr_switches` - Number of context switches

BEST PRACTICES

- ✓ Keep context switches per second as low as possible.
- ✓ Use the right number of threads for the workload.
- ✓ Avoid busy-wait loops.
- ✓ Reduce lock contention and unnecessary wakeups.
- ✓ Monitor regularly and baseline normal behavior.

COMMON CAUSES OF HIGH CONTEXT SWITCHING

- Too many threads or processes
- Lock contention
- Short CPU time slices
- I/O wait with many wakeups
- Poor application design

COMMON MISTAKES

- ✗ Adding more threads to "fix" performance.
- ✗ Ignoring run queue length.
- ✗ Not checking involuntary vs voluntary switches.
- ✗ Blaming CPU without deeper analysis.
- ✗ Not correlating with workload.

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

NUMA AND MULTI-SOCKET PERFORMANCE

QUESTIONS 276-280

276 What is Non-Uniform Memory Access?

277 Why is local memory faster than remote memory on NUMA systems?

278 How can NUMA imbalance degrade database or application performance?

279 How do you inspect NUMA topology and allocation?

280 When should CPU and memory binding be used?

WHAT IS NUMA?

- NUMA (Non-Uniform Memory Access) is an architecture used in multi-socket (multi-CPU) systems.
- Each CPU socket has its own local memory.
- Accessing local memory is faster than accessing memory attached to another socket.
- The memory access time depends on the location of the memory relative to the CPU.

Key Point:
NUMA is about distance. The farther the memory from the CPU, the higher the latency and the lower the bandwidth.

LOCAL vs REMOTE MEMORY ACCESS

ACCESS TYPE	LATENCY	BANDWIDTH	COST
LOCAL (Within same socket)	LOW	HIGH	BEST
REMOTE (Other socket)	HIGH	LOW	EXPENSIVE

Why Local Memory is Faster

- Data travels a shorter path.
- Fewer hops across the interconnect.
- Less contention on the inter-socket links.

IMPACT OF NUMA IMBALANCE

When processes or threads access more remote memory:

- Higher memory latency
- Lower memory bandwidth
- Increased CPU waiting time
- More context switching
- Degraded database performance (e.g., slower lock waits)
- Reduced application throughput

Examples

- Database memory allocated on the wrong node.
- Threads running on CPU 0 but using memory from node 1.
- JVM heap not bound to local memory.

WHEN TO USE CPU AND MEMORY BINDING

Use binding when:

- You run performance-critical workloads.
- You see high remote memory usage.
- You want consistent low-latency performance.
- You run databases (e.g., PostgreSQL, MySQL, Oracle, SQL Server, SAP, Redis).
- You run JVM or in-memory applications.
- You want predictable performance on multi-socket servers.

Key Point:
Binding is a tuning tool. Use it after measuring, not as a default.

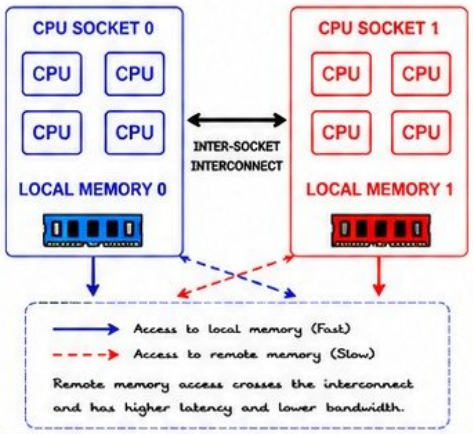
HOW TO INSPECT NUMA TOPOLOGY AND ALLOCATION

1. Understand your hardware (topology).
2. Check how memory is allocated (local vs remote).
3. Identify NUMA imbalance.
4. Decide on binding strategy.
5. Apply and re-measure.
6. Monitor and adjust.

COMMANDS AND TOOLS

COMMAND	PURPOSE	EXAMPLE / NOTES
`lscpu`	Show CPU sockets, cores, threads, NUMA nodes	`lscpu`
`numactl --hardware`	Show NUMA topology and memory info	`numactl --hardware`
`numastat`	Show NUMA memory statistics	`numastat`
`numactl --show`	Show NUMA policy for current process	`numactl --show`
`taskset`	Bind a process to specific CPUs	`taskset -c 0-7 ./app`
`hwloc-ls` (if installed)	Show detailed hardware topology	`hwloc-ls`
`numactl --membind`	Bind memory allocation to a specific node	`numactl --membind=0 ./app`
`numactl --cpunodebind`	Bind CPUs to a specific node	`numactl --cpunodebind=0 ./app`
`numactl --interleave`	Interleave memory across nodes	`numactl --interleave=all ./app`

NUMA ARCHITECTURE DIAGRAM



EXAMPLE: POOR vs GOOD PLACEMENT

POOR (NUMA IMBALANCE)

- Application threads run on CPU socket 0.
- Memory allocated mostly on node 1.
- Every memory access is remote.
- Result: High latency, low throughput.

CPU SOCKET 0 (Threads) → REMOTE ACCESS (SLOW) → MEMORY NODE 1 (Most Memory)

GOOD (NUMA BALANCED)

- Application threads run on CPU socket 0.
- Memory allocated on node 0 (local).
- Most memory accesses are local.
- Result: Low latency, high throughput.

CPU SOCKET 0 (Threads) → LOCAL ACCESS (FAST) → MEMORY NODE 0 (Local Memory)

BEST PRACTICES

- ✓ Keep memory as local as possible.
- ✓ Bind CPUs and memory for critical applications.
- ✓ Use `numactl` or cgroup `cpuset` and memory policies.
- ✓ Monitor with `numastat` and profile performance.
- ✓ Test before and after binding.

COMMON NUMA PITFALLS

- Ignoring NUMA on multi-socket servers.
- Allowing OS to allocate memory anywhere.
- Migrations between nodes (high page migrations).
- NUMA-unaware JVM or database configuration.
- Over-subscribing remote memory bandwidth.

COMMON MISTAKES

- ✗ Assuming more RAM fixes performance.
- ✗ Not checking remote memory usage.
- ✗ Binding without measuring first.
- ✗ Mixing workloads without isolation.
- ✗ Ignoring NUMA in virtualization environments.

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

PERFORMANCE BASELINES AND CAPACITY SIGNALS

QUESTIONS 281–285

281 What is a performance baseline?

282 Which CPU, memory, process and service metrics should be baselined?

283 Why is a single performance snapshot often misleading?

284 How do you distinguish normal workload growth from abnormal degradation?

285 How should baselines influence alert thresholds and capacity planning?

WHAT IS A PERFORMANCE BASELINE?

A performance baseline is a statistical representation of how a system behaves under normal, healthy conditions.

It provides the reference point used to:

- ✓ Detect abnormal behavior
- ✓ Set meaningful alert thresholds
- ✓ Guide capacity planning
- ✓ Measure the impact of changes
- ✓ Track long-term trends

Key Point:
You cannot manage what you do not measure, and you cannot improve what you do not understand.

BASELINE METRICS TO TRACK

CATEGORY	METRIC	DESCRIPTION
CPU	CPU Utilization (%)	Average and per-core usage
	Load Average	System load over 1, 5, 15 min
	Run Queue Length	Number of runnable tasks
	Context Switches	Per second (voluntary + involuntary)
MEMORY	Memory Available	Free memory available to apps
	Swap Activity	Swap in/out and usage
	Page Faults	Major and minor faults
PROCESSES	Total Process Count	Overall number of processes
	Per-Service Process Count	Processes per critical service
	Thread Count	Total and per critical process
SERVICES	Service Latency	Response time (p50, p95, p99)
	Error Rate	Requests failing / total requests
	Restart Frequency	Restarts per time period
	Throughput	Requests or transactions per second

WHY A SINGLE SNAPSHOT IS MISLEADING

- ✗ Workloads fluctuate by time of day, day of week and month.
- ✗ A snapshot may capture a quiet moment.
- ✗ Bursty traffic hides real patterns.
- ✗ Seasonality and business cycles are missed.
- ✗ One value cannot show trend, variance or distribution.

Remember: Trends tell the truth. Snapshots only tell a story.

NORMAL GROWTH vs ABNORMAL DEGRADATION

NORMAL GROWTH	ARRORMAL DEGRADATION
✓ Gradual increase over time	✗ Sudden spikes or jumps
✓ Proportional resource usage	✗ Disproportionate resource usage
✓ Stable latency and error rate	✗ Rising latency or errors
✓ Predictable patterns	✗ Unusual patterns
✓ Capacity remains adequate	✗ Capacity becomes insufficient

The key is correlation: metrics + time + workload.

HOW BASELINES GUIDE ALERTS AND CAPACITY

ALERTING

- Set thresholds based on baseline ranges.
- Use dynamic thresholds where possible.
- Reduce false positives and alert fatigue.

CAPACITY PLANNING

- Understand current headroom.
- Project future growth.
- Plan upgrades before hitting limits.
- Validate changes and new deployments.

PERFORMANCE BASELINE WORKFLOW

1. **COLLECT NORMAL DATA** — Collect metrics over a sufficient period under normal operating conditions.
2. **ESTABLISH RANGES** — Calculate averages, medians, percentiles and standard deviation.
3. **TRACK TRENDS** — Monitor metrics continuously and visualize trends.
4. **DETECT DEVIATION** — Identify metrics outside expected ranges or showing unusual patterns.
5. **INVESTIGATE** — Correlate with logs, changes, deployments and workload.
6. **UPDATE BASELINE** — Adjust baseline as the system and workload evolve.

EXAMPLE BASELINE RANGES (PRODUCTION WEB SERVER)

METRIC	TYPICAL RANGE	WARNING LEVEL	CRITICAL LEVEL
CPU Utilization (avg)	15% - 60%	60% - 80%	> 80%
Load Average (1m)	0.5 - 2.0	2.0 - 4.0	> 4.0
Run Queue Length	0 - 2	3 - 5	> 5
Context Switches (/s)	5k - 20k	20k - 40k	> 40k
Memory Available	30% - 50%	15% - 30%	< 15%
Swap Activity (in+out/s)	0 - 10	10 - 50	> 50
Process Count	150 - 400	400 - 700	> 700
Service Latency (p95)	50 ms - 200 ms	200 ms - 500 ms	> 500 ms
Error Rate	0% - 0.5%	0.5% - 2%	> 2%
Restart Frequency	0 - 2 / day	2 - 5 / day	> 5 / day

DATA COLLECTION BEST PRACTICES

- ✓ Collect for at least 1–2 weeks.
- ✓ Include peak and off-peak periods.
- ✓ Use consistent intervals (e.g., 1 min).
- • Tag metrics with environment and service.
- • Store data centrally (TSDB or metrics DB).
- • Use dashboards for visualization.
- • Review baselines regularly.
- ✓ Document assumptions and changes.

COMMON PITFALLS

- ✗ Setting thresholds without baseline data.
- ✗ Ignoring percentiles (p90, p95, p99).
- ✗ Using static thresholds for dynamic systems.
- ✗ Not accounting for business seasonality.
- ✗ Not updating baselines as workload evolves.
- ✗ Alerting on symptoms, not root causes.

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

PRODUCTION PERFORMANCE TOOLKIT

QUESTIONS 286-290

286. When should you use top, htop, ps or pidstat?
287. What does vmstat reveal that top may not show clearly?
288. How do mpstat and sar support historical performance analysis?
289. How do you correlate CPU, memory, process and I/O data?
290. What investigation order prevents misleading conclusions?

TOOLKIT OVERVIEW

Linux provides many powerful tools. The key is knowing what each tool is best for and how they work together.

- ✓ No single tool tells the whole story.
- ✓ Combine live data + trends + history.
- ✓ Always correlate across CPU, memory, processes, I/O and services.
- ✓ Base conclusions on evidence, not assumptions.

Key Point:
Good engineers collect data. Great engineers correlate data. Senior engineers understand cause and effect.

TOOL COMPARISON

TOOL	PRIMARY PURPOSE	BEST USED FOR
top	Live system overview	Quick health check, find busy processes
htop	Interactive process analysis	Drill down into processes, threads and CPU
ps	Point-in-time process snapshot	Evidence collection, point-in-time state
pidstat	Per-process performance trends	Track CPU, memory, I/O per process
vmstat	System pressure and resource summary	Understand run queue, swap, I/O and memory
mpstat	Per-CPU activity	Identify CPU hotspots, imbalance and idle
sar	Historical system statistics	Trend analysis, capacity planning, incidents

WHEN TO USE EACH TOOL

- top – When you need a real-time overview of CPU, memory, processes and load.
- htop – When you want an interactive view, tree view, and easy process management.
- ps – When you need a snapshot of processes for logging, scripting or evidence.
- pidstat – When you need per-process CPU, memory, I/O trends over time.
- vmstat – When you suspect system pressure (run queue, swap, I/O wait) but top looks "normal".
- mpstat – When you suspect CPU imbalance or want to see per-CPU utilization.
- sar – When you need historical data and long-term trends.

WHAT VMSTAT REVEALS THAT TOP MAY HIDE

- Run queue length (r) – shows contention.
- Blocked tasks (b) – tasks waiting for I/O.
- Swap activity (si/so) – swap in/out.
- I/O performance (bi/bo) – block I/O rates.
- Context switches (cs) – system workload.
- Interrupts (in) – hardware and software interrupts.
- CPU idle time across all cores.

```
# vmstat 1
procs -----------memory---------- ---swap-- -----io---- -system-- -----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 2  1    512 102400 20480 512000    0    0   120    80 3000 5000  8  6 84  2  0
 4  2    512  99000 20480 520000    0    0   350   250 3200 5600 25 15 55  5  0
```

High r or b with high wa usually means I/O is the bottleneck, not the CPU.

HOW MPSTAT AND SAR HELP

mpstat – Real-time per-CPU view

- Shows utilization (user, system, idle, iowait) for each CPU.
- Helps detect CPU hotspots and imbalance.
- Example: `mpstat -P ALL 1`

sar – Historical and trending data

- Collects and reports system activity over time.
- Useful for:
  - Capacity planning
  - Performance baselines
  - Incident analysis
  - Long-term trend identification

Example: `sar -u 1 60` (CPU every 1 sec for 60 sec)

CORRELATING CPU, MEMORY, PROCESS AND I/O DATA

- CPU – High CPU? Check which process. Check run queue and CPU wait.
- MEMORY – Low available memory? Check cache, swap activity and memory per process.
- PROCESSES – High load? Which processes or threads are causing it? Are they expected?
- I/O – High I/O wait or blocked tasks? Check disk latency, I/O rates and queues.

CORRELATION EXAMPLE

High load average (r)
+
High I/O wait (wa)
+
Many blocked tasks (b)
=
Storage bottleneck

Always correlate, never assume.

INVESTIGATION ORDER (PREVENTS MISLEADING CONCLUSIONS)

1. Symptom Identification – What is the user or system experiencing?
2. System Overview (Live) – Use top or htop to get a quick overview.
3. Resource Pressure Check – Use vmstat to check run queue, swap, I/O wait.
4. Process Attribution – Use pidstat, ps, htop to identify the culprit.
5. Historical Comparison – Use sar and mpstat to compare with past data.
6. Root Cause Determination – Correlate all data and confirm the cause.

Important

Skipping steps leads to wrong conclusions.

Example:
High CPU?
It might be:
- CPU bound
- I/O bound
- Lock contention
- Memory pressure
- External dependency

Follow the order.

COMMON COMMAND QUICK REFERENCE

COMMAND	PURPOSE
top	Real-time system overview
htop	Interactive process analysis
ps -eo pid,ppid,pcpu,pmem,cmd	Process snapshot with CPU & memory
pidstat -u -r -d -w 1	Per-process CPU, memory, I/O, context switches
vmstat 1	System performance and pressure
mpstat -P ALL 1	Per-CPU utilization
sar -u 1 60	Historical CPU statistics
sar -r 1 60	Historical memory statistics
sar -d 1 60	Historical disk I/O statistics

BEST PRACTICES

- ✓ Always establish a performance baseline.
- ✓ Collect data over time, not just snapshots.
- ✓ Correlate across all resource domains.
- ✓ Use multiple tools to confirm findings.
- ✓ Document findings and root cause.
- ✓ Review and update baselines regularly.

COMMON MISTAKES

- ✗ Relying on a single metric or tool.
- ✗ Reacting without collecting evidence.
- ✗ Ignoring I/O wait and run queue.
- ✗ Not comparing with historical data.
- ✗ Jumping to conclusions.

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer



SERVICE AND PERFORMANCE INCIDENT SCENARIOS

QUESTIONS 291–295

291. A service repeatedly restarts every few seconds. How do you investigate it?
292. CPU is low, but load average is extremely high. What do you check?
293. Memory usage grows gradually until the service is killed. How do you prove the cause?
294. One CPU core is saturated while the others remain idle. What could explain it?
295. A service is running, but users still receive errors. How do you separate service health from application health?

REQUIRED INCIDENT ANSWER STRUCTURE

1. Confirm impact – What is affected and how many users?
2. Establish timeline – When did it start and what changed?
3. Gather system and service evidence – Logs, metrics, processes, resources, dependencies.
4. Form hypotheses – List the most likely causes.
5. Test safely – Validate each hypothesis with minimal risk.
6. Recover service – Restore normal operation.
7. Verify stability – Monitor and ensure full recovery.
8. Prevent recurrence – Implement fixes and improvements.

Key Point:
Great engineers follow a repeatable process.
This prevents panic and shortens downtime.

INCIDENT INVESTIGATION PRINCIPLES

- ✓ Collect data before making changes.
- ✓ Correlate metrics, logs and events.
- ✓ Look for the first failure, not the last failure.
- ✓ Differentiate between symptoms and root causes.
- ✓ Always consider dependencies and external systems.
- ✓ Document everything during the incident.
- ✓ Fix the cause, not just the immediate symptom.

NEVER DO THESE:

- ✗ Restart blindly without understanding the cause.
- ✗ Ignore historical patterns or recent changes.
- ✗ Rely on a single metric or tool.
- ✗ Assume the problem without evidence.
- ✗ Skip post-incident review.

COMMON SERVICE AND PERFORMANCE INCIDENT SCENARIOS

1 Service Restart Loop (Every Few Seconds)

1. Confirm impact (users, errors).
2. Check service status and restart count.
3. Review logs: journalctl -u <service>.
4. Check recent changes and config.
5. Verify dependencies (DB, network, files).
6. Use systemd-analyze for startup time.
7. Fix root cause and stabilize.
8. Monitor and prevent recurrence.

Tools:
systemctl status, journalctl -u, systemd-analyze, lsof, strace

2 High Load Average, Low CPU

1. Confirm impact and symptoms.
2. Check load average and run queue.
3. Identify uninterruptible processes (D state).
4. Check I/O wait: vmstat 1, iostat -xz 1.
5. Check disk, storage, network, DB.
6. Identify the blocking resource.
7. Resolve the bottleneck.
8. Verify recovery.

Tools:
top, vmstat, iostat, pidstat -w, lsof, strace

3 Memory Growth Until Killed

1. Confirm impact and OOM events.
2. Review OOM killer logs (dmesg, journalctl -k).
3. Check memory trend over time (sar -r).
4. Identify process memory usage (smem, pmap).
5. Analyze leaks or unbounded caches.
6. Fix memory issue or set limits.
7. Validate stability.
8. Prevent recurrence (code/config/limits).

Tools:
free -h, vmstat 1, smem, pmap, journalctl -k, sar -r, systemd-cgtop

4 One CPU Core 100%, Others Idle

1. Confirm CPU saturation on a single core.
2. Identify the process and thread.
3. Check CPU affinity and pinning.
4. Check mutex/lock contention.
5. Check single-threaded bottlenecks.
6. Profile with perf/top or strace.
7. Optimize code or adjust affinity.
8. Rebalance workload.

Tools:
top, htop, ps -eLo pid,psr,cmd,%cpu, pidstat -u -t 1, perf top

5 Service Running, But Users See Errors

1. Confirm service is running and healthy.
2. Check application logs and error rates.
3. Validate dependencies (DB, cache, API).
4. Check network, DNS, load balancer.
5. Measure latency and response time.
6. Correlate errors with metrics and logs.
7. Identify root cause.
8. Fix and verify end-to-end health.

Tools:
journalctl -u, curl, netstat/ss, strace, lsof, application logs

INCIDENT TOOLS QUICK REFERENCE

TOOL	PURPOSE	EXAMPLE USAGE
systemctl status	Check service status and health	systemctl status nginx
journalctl -u	View service logs	journalctl -u nginx -xe
systemd-analyze	Analyze startup time and dependencies	systemd-analyze blame
top	Live process and resource overview	top
pidstat	Per-process CPU, memory, I/O, threads	pidstat -u -r -d 1
vmstat	System performance and pressure	vmstat 1
mpstat	Per-CPU utilization and stats	mpstat -P ALL 1
sar	Historical system statistics	sar -u -r -d 1 60
strace	Trace system calls of a process	strace -p <PID> -f
lsof	List open files and network connections	lsof -i -P -n

SAMPLE TIMELINE CORRELATION

TIME	EVENT	METRIC IMPACT
10:00	Deployment started	Normal
10:05	Service restart loop begins	Errors ↑
10:06	DB connection pool exhausted	Load ↑, I/O wait ↑
10:07	Users experience errors	Latency ↑, Errors ↑
10:10	Root cause identified	Stabilizing
10:15	Fix applied	Recovery
10:20	Service stable	Normal

Key Point:
Correlate events, metrics and logs on a timeline.
It reveals the true root cause.

POST-INCIDENT CHECKLIST

- ✓ Document root cause and contributing factors.
- ✓ Update alerts and dashboards.
- ✓ Implement fixes and safeguards.
- ✓ Review runbooks and procedures.
- ✓ Share learnings with the team.
- ✓ Validate stability under normal load.
- ✓ Schedule follow-up review.

VERIQTA

LINUX PRODUCTION INTERVIEW BIBLE

SECTION 2: PROCESSES, SERVICES AND PERFORMANCE

Think Like a Senior Production Engineer

SECTION 2 PRODUCTION REVIEW

QUESTIONS 296–300

296. How would you investigate a production host experiencing high load, memory pressure and service restarts simultaneously?
297. How do you determine whether a performance issue originates from the application, kernel, scheduler or infrastructure?
298. How would you design safe resource limits for a critical service?
299. How do you create a performance baseline and alerting strategy for a Linux production server?
300. How would you lead a complete Linux performance incident from detection through postmortem?

INVESTIGATION FRAMEWORK

1. Confirm Impact – What is affected and how many users?
2. Establish Timeline – When did it start and what changed?
3. Gather Evidence – Logs, metrics, processes, resources, dependencies.
4. Form Hypotheses – List the most likely causes.
5. Test Safely – Validate each hypothesis with minimal risk.
6. Recover Service – Restore normal operation.
7. Verify Stability – Monitor and ensure full recovery.
8. Prevent Recurrence – Implement fixes and improvements.

Key Point:
Follow a repeatable process. Great engineers collect evidence before they make changes. This prevents downtime and guesswork.

SENIOR ENGINEER INSIGHTS

- ✓ Start with impact, not commands.
- ✓ Correlate several metrics before drawing conclusions.
- ✓ Understand the relationship between processes, services, kernel resources and infrastructure.
- ✓ Capture evidence before restarting or killing anything.
- ✓ Prefer reversible mitigation during an active incident.
- ✓ Separate immediate recovery from permanent remediation.
- ✓ Measure the result of every performance change.

REMEMBER

Evidence → Correlation → Hypothesis → Validation → Resolution → Improvement

PROCESSES AND SERVICES CHEAT SHEET

Processes

- Created by the kernel.
- Consume CPU, memory, I/O.
- Have a parent (PPID).
- Run in states (R, S, D, Z, T).

Services (systemd)

- Managed by systemd.
- Have unit files.
- Can auto-restart.
- Depend on other units.
- Have resource limits.



PROCESS INSPECTION COMMANDS

COMMAND	PURPOSE
ps -eo pid,ppid,stat,cmd,%cpu,%mem	Snapshot of all processes
ps aux	Detailed process list
top	Live process and system view
htop	Interactive process analysis
pidstat -u -t 1	Per-process CPU and threads
ps -eo pid,tid,psr,%cpu,stat,comm	Per-thread CPU and core
lsof -p <PID>	Open files for a process
cat /proc/<PID>/status	Process details and limits
pmap -x <PID>	Memory map of a process
smem -r	Detailed memory usage

PROCESS STATE REFERENCE (STAT COLUMN)

STATE	MEANING	DESCRIPTION
R	Running	Actively running or ready to run
S	Sleeping	Interruptible sleep (waiting)
D	Uninterruptible Sleep	Waiting for I/O, cannot be interrupted
T	Stopped	Stopped by job control signal
t	Tracing Stop	Stopped by debugger
Z	Zombie	Exited but not reaped by parent
I	Idle	Kernel idle task

SYSTEMD INVESTIGATION COMMANDS

COMMAND	PURPOSE
systemctl status <service>	Service status and recent logs
systemctl list-units --type=service	List all services
systemctl list-dependencies <service>	Show unit dependencies
journalctl -u <service>	Service logs
journalctl -xe	System logs (errors first)
systemd-analyze blame	Slowest unit startups
systemd-analyze critical-chain	Startup dependency chain
systemd-analyze verify <unit>	Validate unit file
systemctl show <service>	All unit properties
systemctl cat <service>	Show unit file contents

UNIT FILE DIRECTIVES (IMPORTANT)

DIRECTIVE	PURPOSE
ExecStart=	Command to start
ExecStop=	Command to stop
Restart=	Restart policy
RestartSec=	Delay before restart
StartLimitIntervalSec=	Time window for restart limits
StartLimitBurst=	Max restarts in interval
TimeoutStartSec=	Max time to start
TimeoutStopSec=	Max time to stop
LimitNOFILE=	File descriptor limit
MemoryMax=	Memory limit (cgroup v2)
CPUQuota=	CPU limit (cgroup v2)
CPUWeight=	CPU share weight

RESTART CONTROLS

DIRECTIVE	DESCRIPTION
Restart=no	Do not restart automatically
Restart=on-failure	Restart if exited with error
Restart=always	Restart regardless of exit code
Restart=on-abnormal	Restart on signal or timeout
RestartSec=5s	Wait 5 seconds before restart
StartLimitIntervalSec=60	Restart limit window (seconds)
StartLimitBurst=3	Max 3 restarts in the window

Protection: Use StartLimit* to prevent restart loops and cascading failures.

RESOURCE LIMIT COMMANDS

COMMAND	PURPOSE
ulimit -a	View current limits
ulimit -n <65535	Set open file limit
prlimit --pid <PID>	View/modify process limits
cat /proc/<PID>/limits	View process limits
cat /sys/fs/cgroup/<?/"	View cgroup limits (v2)
systemd-run --scope -p MemoryMax=1G <cmd>	Run with memory limit
systemd-run --scope -p CPUQuota=50% <cmd>	Run with CPU limit
systemd-cgtop	Live cgroup resource usage

PERFORMANCE COMMAND REFERENCE

COMMAND	PURPOSE
top	Live overview (CPU, MEM, PROCS)
htop	Interactive live analysis
ps aux	Process snapshot
pidstat 1	Per-process trends
vmstat 1	System pressure overview
mpstat -P ALL 1	Per-CPU activity
sar -u 1	Historical CPU usage
sar -r 1	Historical memory usage
sar -d 1	Historical I/O usage
free -h	Memory summary
numastat	NUMA allocation stats
systemd-cgtop	Cgroup live usage

COMMON PRODUCTION MISTAKES

- ✗ Killing processes before gathering evidence.
- ✗ Treating load average as CPU usage.
- ✗ Ignoring uninterruptible processes (D state).
- ✗ Using Restart=always without rate limits.
- ✗ Increasing limits without understanding consumption.
- ✗ Assuming cached memory is wasted memory.
- ✗ Ignoring per-thread CPU usage.
- ✗ Tuning swappiness without measuring memory pressure.
- ✗ Tuning CPU before profiling the workload.
- ✗ Using one performance snapshot as the root cause.
- ✗ Making changes without recording a baseline.
- ✗ Failing to monitor the system after recovery.

PRODUCTION READINESS CHECKLIST

PROCESSES

- ☐ Process ownership documented
- ☐ Expected parent-child relationships understood
- ☐ Process counts monitored
- ☐ Zombie growth alerts configured
- ☐ Critical processes whitelisted from OOM

SERVICES

- ☐ Unit files validated
- ☐ Dependencies documented
- ☐ Restart policies controlled
- ☐ Startup and shutdown timeouts configured
- ☐ Logs retained and searchable

RESOURCES

- ☐ File descriptor limits reviewed
- ☐ Memory limits tested
- ☐ CPU allocations documented
- ☐ Cgroup policies monitored
- ☐ Disk, network and inode capacity monitored

PERFORMANCE

- ☐ Baselines established
- ☐ Historical metrics retained
- ☐ CPU, memory and load alerts configured
- ☐ Capacity trends reviewed
- ☐ Incident runbooks tested

INCIDENT PREPAREDNESS

- ☐ Runbooks exist and updated
- ☐ On-call team trained
- ☐ Escalation paths defined
- ☐ Communication templates ready
- ☐ Postmortem template ready